GOOGLE CLOUD

CHEAT SHEETS

Google Cloud for developers and operators, from gcloud to Gemini

gcloud · Projects · IAM · Service Accounts · Workload Identity Federation · Secret Manager · Cloud KMS · VPC · Firewall Policies · Cloud NAT · Cloud DNS · Cloud CDN · Compute Engine · MIGs · Load Balancing · Cloud Run · Pub/Sub · Eventarc · Artifact Registry · GKE · Cloud Storage · Cloud SQL · Firestore · BigQuery · Terraform · Cloud Logging · Agent Platform · Gemini

42 REFERENCE SHEETS

CONTENTS

GETTING STARTED

01 Overview & Global Infrastructure
02 Projects & Billing
03 gcloud CLI
04 Client Libraries & Credentials

IDENTITY & SECURITY

05 IAM Principals & Roles
06 IAM Policies & Conditions
07 Service Accounts & Workload Identity Federation
08 Secret Manager & Cloud KMS
09 Security Services

NETWORKING

10 VPC Basics
11 Cloud NAT & Connectivity
12 Firewall Rules & Policies
13 Cloud DNS
14 Cloud CDN & Certificates

COMPUTE

15 Compute Engine Basics
16 Compute Engine Operations & Pricing

SERVERLESS & MESSAGING

20 Cloud Run Basics
21 Cloud Run Functions & Jobs
22 API Gateway & Apigee
23 Eventarc, Workflows & Scheduler
24 Pub/Sub

CONTAINERS

25 Artifact Registry
26 GKE Basics
27 GKE Operations

STORAGE

28 Cloud Storage Basics
29 Cloud Storage Security & Features
30 Disks, Filestore & Backup

DATABASES

31 Cloud SQL & AlloyDB
32 Firestore, Spanner & Bigtable
33 BigQuery & Memorystore

INFRASTRUCTURE AS CODE

34 Terraform on Google Cloud
35 Infrastructure Manager
36 Config Connector & Choosing IaC

OBSERVABILITY & DEPLOYMENT

37 Cloud Monitoring
38 Cloud Logging & Trace
39 Deploying to Google Cloud

AI & MACHINE LEARNING

40 Agent Platform (Vertex AI) & Gemini
41 AI & ML Services

QUICK REFERENCE

42 Quick Reference — gcloud & Service Picker

GCP

Table of Contents

42 sheets · syntax · examples · notes

GETTING STARTED 01 – 04

- 01 Overview & Global Infr…
- 02 Projects & Billing
- 03 gcloud CLI
- 04 Client Libraries & Cred…

IDENTITY & SECURITY 05 – 09

- 05 IAM Principals & Roles
- 06 IAM Policies & Conditi…
- 07 Service Accounts & W…
- 08 Secret Manager & Clou…
- 09 Security Services

NETWORKING 10 – 14

- 10 VPC Basics
- 11 Cloud NAT & Connecti…
- 12 Firewall Rules & Policies
- 13 Cloud DNS
- 14 Cloud CDN & Certifica…

COMPUTE 15 – 19

- 15 Compute Engine Basics
- 16 Compute Engine Oper…
- 17 Cloud Load Balancing
- 18 Managed Instance Gro…
- 19 Choosing Compute

SERVERLESS & MESSAGING 20 – 24

- 20 Cloud Run Basics
- 21 Cloud Run Functions &…
- 22 API Gateway & Apigee
- 23 Eventarc, Workflows & …
- 24 Pub/Sub

CONTAINERS 25 – 27

- 25 Artifact Registry
- 26 GKE Basics
- 27 GKE Operations

STORAGE 28 – 30

- 28 Cloud Storage Basics
- 29 Cloud Storage Securit…
- 30 Disks, Filestore & Back…

DATABASES 31 – 33

- 31 Cloud SQL & AlloyDB
- 32 Firestore, Spanner & Bi…
- 33 BigQuery & Memoryst…

INFRASTRUCTURE AS CODE 34 – 36

- 34 Terraform on Google C…
- 35 Infrastructure Manager
- 36 Config Connector & C…

OBSERVABILITY & DEPLOYMENT 37 – 39

- 37 Cloud Monitoring
- 38 Cloud Logging & Trace
- 39 Deploying to Google Cl…

AI & MACHINE LEARNING 40 – 41

- 40 Agent Platform (Vertex…
- 41 AI & ML Services

QUICK REFERENCE 42

- 42 Quick Reference — gcl…

Sheet 01

Overview & Global Infrastructure

regions & zones · global vs regional vs zonal resources · Premium vs Standard network tier · organization, folders & projects · project ID vs number · Free Tier & free trial

GOOGLE'S CLOUD

Google Cloud runs in 40+ **regions**, each with three or more **zones**, connected by Google's private global network. Resources are **zonal** (VMs, disks), **regional** (subnets, Cloud Run services, regional buckets) or **global** (VPC networks, global load balancers, IAM). Everything lives in a **project**, grouped under **folders** and an **organization**.

KEY POINTS

- Region / zone names: `us-central1`, `us-central1-a`, `europe-west1-b`
- A project has a chosen, permanent **project ID** and a numeric **project number**
- APIs are disabled by default per project: enable what you use
- Premium network tier rides Google's backbone; Standard uses the public internet

EXPLORE PROJECTS, REGIONS, ZONES

```
gcloud auth login
gcloud projects list --format="table(projectId,projectNumber,name)"
gcloud config set project shop-prod-4821
gcloud organizations list
gcloud resource-manager folders list --organization=123456789012

gcloud compute regions list --format="value(name)"
gcloud compute zones list --filter="region:us-central1" --format="value(name)"
gcloud services list --enabled          # APIs turned on in this project
gcloud services enable run.googleapis.com compute.googleapis.com

# Full resource names identify everything
# //compute.googleapis.com/projects/shop-prod-4821/zones/us-central1-a/
#   instances/web-1
gcloud asset search-all-resources --scope=projects/shop-prod-4821 \
    --query="labels.env=prod" --format="value(name)"
```

CODE EXPLAINED

- → `gcloud config set project` sets the default for later commands
- → Zones are listed per region with a filter
- → Enable APIs before using a service the first time
- → Cloud Asset Inventory searches resources by label across services

AWS / AZURE EQUIVALENT

- ↔ **AWS** Regions, AZs, accounts, ARNs (aws-01)
- ↔ **Azure** Regions, zones, subscriptions, resource groups (az-01)
- ↔ Google Cloud VPCs are global; AWS and Azure networks are regional

COST CHECK

- $ Free trial credit for 90 days plus Always Free usage (e2-micro, Cloud Run, storage quotas)
- $ Standard network tier costs less for egress but is slower
- $ Inter-zone and inter-region traffic bills per GB

SCOPE

SCOPE	EXAMPLES
Global	VPC networks, IAM, global ALB, Cloud DNS
Regional	Subnets, Cloud Run, Cloud SQL, regional buckets
Zonal	VMs, Persistent Disks, zonal GKE
Multi-region	Some buckets, BigQuery datasets, Spanner configs

HIERARCHY

LEVEL	HOLDS
Organization	Tied to a Workspace / Cloud Identity domain
Folder	Projects or folders; policy inheritance
Project	Resources, APIs, billing link
Resource	VM, bucket, service…

DO & DON'T

- ✓ **Do** Use one project per app per environment
- ✓ **Do** Label resources (team, env, app)
- ✗ **Don't** Put unrelated apps in one project
- ✗ **Don't** Pick a project ID you will regret (it is permanent)

LOCAL TESTING

- ⌂ Cloud Shell has gcloud, kubectl, Terraform and an editor
- ⌂ The free trial plus a budget alert is a safe sandbox
- ⌂ Many services have local emulators (Pub/Sub, Firestore, Bigtable, Spanner)

PRODUCTION CHECKLIST

- ☐ Folders for prod / non-prod with policies attached
- ☐ Organization policies set before teams build
- ☐ Labels enforced for cost attribution

Projects & Billing

creating projects · billing accounts · budgets & alerts · labels · billing export to BigQuery · `gcloud services enable` (APIs are off by default)

PROJECTS, APIS AND PAYING

Each **project** links to one **billing account**. Projects are where APIs are enabled, quotas apply and IAM is granted. **Budgets** alert at thresholds (and can publish to Pub/Sub for automation). **Labels** attribute costs, and **billing export to BigQuery** gives detailed, queryable cost data. Shutting down a project stops its billing.

KEY POINTS

- Deleted projects can be restored for 30 days
- Budgets alert but do not cap spending (automate shutdown via Pub/Sub if needed)
- Quotas are per project (and often per region)
- Use folders to group projects by team or environment

LIMITS & QUOTAS

- Project creation is quota-limited per user / organization (request more)
- Up to 64 labels per resource
- Service quotas per project and region: check in Quotas & System Limits

PROJECTS, BILLING, BUDGETS, QUOTAS

```
gcloud projects create shop-prod-4821 --folder=987654321 \
    --labels=team=web,env=prod
gcloud billing projects link shop-prod-4821 \
    --billing-account=0X0X0X-0X0X0X-0X0X0X
gcloud services enable run.googleapis.com sqladmin.googleapis.com \
    secretmanager.googleapis.com --project=shop-prod-4821

gcloud billing budgets create --billing-account=0X0X0X-0X0X0X-0X0X0X \
    --display-name="shop-prod monthly" --budget-amount=300USD \
    --filter-projects=projects/shop-prod-4821 \
    --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 \
    --threshold-rule=percent=1.0

gcloud projects update shop-prod-4821 --update-labels=costcenter=1234
gcloud compute project-info describe --flatten=quotas \
    --format="table(quotas.metric,quotas.limit,quotas.usage)"
gcloud projects delete shop-scratch-1    # stops billing; 30-day restore
```

CODE EXPLAINED

- Labels set at creation help cost reports
- Billing must be linked before most APIs work
- Thresholds at 50%, 90%, 100% of $300 email billing admins
- `project-info describe` lists Compute Engine quotas and usage

AWS / AZURE EQUIVALENT

↔	**AWS**	Organizations, accounts, AWS Budgets (aws-02)
↔	**Azure**	Subscriptions, Cost Management budgets (az-02)

COST CHECK

- $ Projects, budgets and billing export are free (BigQuery storage / queries bill normally)
- $ Sustained use discounts apply automatically to many VM types
- $ Leftover static IPs, disks and snapshots keep billing

BILLING TOOLS

TOOL	DOES
`Billing reports`	Explore spend
`Budgets & alerts`	Thresholds, Pub/Sub automation
`BigQuery export`	Detailed cost data
`Recommender`	Rightsizing, idle resources
`Pricing calculator`	Estimate before building

PROJECT IDS

ITEM	EXAMPLE
`Project ID`	shop-prod-4821 (you choose, global)
`Project number`	123456789012 (assigned)
`Project name`	Shop Prod (display only)

DO & DON'T

- ✓ **Do** Create a budget per project before deploying
- ✓ **Do** Turn on BigQuery billing export on day one
- ✗ **Don't** Share one project across teams and environments
- ✗ **Don't** Assume a budget alert will stop spending

LOCAL TESTING

- Estimate with the Google Cloud Pricing Calculator
- Infracost reads Terraform google plans
- Practice in a scratch project you delete afterwards

PRODUCTION CHECKLIST

- ☐ Budgets with Pub/Sub alerts per project
- ☐ Billing export to BigQuery with dashboards
- ☐ Label policy enforced via org policy / IaC

gcloud CLI

install · `gcloud init` · `gcloud auth login` vs `application-default login` · configurations · `--project` · `--format` / `--filter` · Cloud Shell · `gcloud storage` replaces `gsutil`

SCRIPTING GOOGLE CLOUD

The **gcloud CLI** manages nearly every Google Cloud service: `gcloud <group> <command>`. **Configurations** hold named sets of account, project and region. `gcloud auth login` signs in the CLI; `gcloud auth application-default login` creates credentials for **your code** (client libraries). `--format` and `--filter` shape output, and `gcloud storage` is the modern replacement for `gsutil`.

KEY POINTS

- Components: `gcloud components install kubectl beta`
- Release tracks: GA, `beta`, `alpha` commands
- `bq` is the BigQuery CLI; `gcloud storage` covers Cloud Storage
- `--impersonate-service-account` runs commands as a service account

SECURITY CHECK

- ⚠ Run automation as service accounts via impersonation or WIF, not key files
- ⚠ Revoke old credentials: `gcloud auth revoke`
- ⚠ Keep `~/.config/gcloud` private; it holds tokens

EVERYDAY GCLOUD

```
gcloud init                                  # first-time setup
gcloud auth login                            # CLI credentials
gcloud auth application-default login        # credentials for code (ADC)

gcloud config configurations create shop-dev
gcloud config set project shop-dev-1234
gcloud config set compute/region us-central1
gcloud config configurations activate shop-prod
gcloud config list

gcloud compute instances list \
    --filter="status=RUNNING AND labels.env=prod" \
    --format="table(name,zone.basename(),machineType.basename())"
gcloud run services list --format="value(metadata.name)"

gcloud storage ls gs://acme-media-prod/2026/
gcloud storage buckets list \
    --impersonate-service-account=ops@shop-prod-4821.iam.gserviceaccount.com

gcloud components update
gcloud help run deploy                       # full docs
```

CODE EXPLAINED

- → Two logins: one for the CLI, one for client libraries
- → Configurations switch account / project / region together
- → `--filter` selects rows; `--format` picks and transforms columns
- → Impersonation borrows a service account's permissions temporarily

AWS / AZURE EQUIVALENT

- ↔ **AWS** AWS CLI profiles, `--query` (aws-03)
- ↔ **Azure** Azure CLI, `az account set`, `--query` (az-03)
- ↔ gcloud uses `--filter` / `--format` instead of JMESPath

COST CHECK

- $ The CLI and Cloud Shell are free
- $ API calls bill the same as from the console
- $ Watch scripts that loop over paid APIs

GLOBAL FLAGS

FLAG	DOES
`--project`	One-off project
`--format=json|yaml|table(…)|value(…)`	Output
`--filter`	Server / client filter
`--impersonate-service-account`	Act as SA
`--quiet / -q`	No prompts
`--verbosity=debug`	Debug output

OLD → NEW

OLD	NEW
`gsutil cp`	gcloud storage cp
`gsutil rsync`	gcloud storage rsync
`gcloud beta run`	gcloud run (GA)
`Deployment Manager`	Infrastructure Manager / Terraform

DO & DON'T

- ✓ **Do** Use configurations per project / environment
- ✓ **Do** Pass `--project` explicitly in scripts
- ✗ **Don't** Download service account keys for your laptop
- ✗ **Don't** Parse table output in scripts (use `value(...)` or JSON)

LOCAL TESTING

- ⌂ Cloud Shell needs no install
- ⌂ Docker image `gcr.io/google.com/cloudsdktool/google-cloud-cli`
- ⌂ `gcloud beta emulators` starts Pub/Sub, Firestore, Bigtable, Spanner emulators

PRODUCTION CHECKLIST

- ☐ Automation uses WIF or attached service accounts
- ☐ Scripts pin project and region, fail on errors
- ☐ Cloud Audit Logs capture all CLI actions

Sheet 04

Client Libraries & Credentials

Application Default Credentials · lookup order · Python & Node.js client libraries · service account impersonation · metadata server · avoiding key files

AUTHENTICATING CODE

Google Cloud client libraries use **Application Default Credentials (ADC)**: the `GOOGLE_APPLICATION_CREDENTIALS` file if set, your `gcloud auth application-default login` credentials on a laptop, or the **attached service account** via the **metadata server** on Google Cloud compute. The same code works everywhere with no keys in the code.

KEY POINTS

- Python: `pip install google-cloud-storage`; Node: `npm i @google-cloud/storage`
- ADC can impersonate a service account locally (`--impersonate-service-account`)
- Quota project is set with `gcloud auth application-default set-quota-project`
- Prefer WIF configs over key files outside Google Cloud

SECURITY CHECK

- ⚠ Block service account key creation with org policy
- ⚠ Attach a dedicated service account to each workload
- ⚠ Never commit JSON key files; scan repos for them

PYTHON AND NODE.JS

```python
from google.cloud import storage, secretmanager
gcs = storage.Client()                    # ADC: laptop login or attached SA
for b in gcs.list_buckets():
    print(b.name)

sm = secretmanager.SecretManagerServiceClient()
name = "projects/shop-prod-4821/secrets/db-password/versions/latest"
password = sm.access_secret_version(name=name).payload.data.decode()
```

CODE EXPLAINED

- → No credentials appear in code: ADC finds them
- → Secret Manager returns bytes; decode to text
- → The Node client uses the same ADC chain
- → Impersonated ADC tests with the app's real permissions

NODE.JS

```node
import { Storage } from "@google-cloud/storage";
const storage = new Storage();            // ADC
const bucket = storage.bucket("acme-media-prod");
const [files] = await bucket.getFiles({ prefix: "2026/" });
console.log(files.map(f => f.name));
```

AWS / AZURE EQUIVALENT

- ↔ **AWS** AWS credential provider chain (aws-04)
- ↔ **Azure** DefaultAzureCredential (az-04)

LOCAL ADC OPTIONS

```
gcloud auth application-default login
gcloud auth application-default login \
    --impersonate-service-account=api@shop-dev-1234.iam.gserviceaccount.com
```

COST CHECK

- $ Client libraries are free; you pay for API usage
- $ Metadata server token requests are free
- $ Reusing clients avoids extra connection and auth overhead

ADC LOOKUP ORDER

ORDER	SOURCE
1	GOOGLE_APPLICATION_CREDENTIALS (file or WIF config)
2	gcloud application-default login
3	Attached service account (metadata server)

PICK A METHOD

WHERE	USE
Laptop	application-default login (+ impersonation)
Cloud Run / Functions / VMs	Attached service account
GKE	Workload Identity Federation for GKE
GitHub Actions / other clouds	Workload Identity Federation
On-premises	WIF config file

DO & DON'T

- ✓ **Do** Create one client per process and reuse it
- ✓ **Do** Use impersonation locally to test least privilege
- ✗ **Don't** Download service account keys
- ✗ **Don't** Use your personal user credentials in production

LOCAL TESTING

- ⌂ ADC from `gcloud auth application-default login`
- ⌂ Emulators: set `PUBSUB_EMULATOR_HOST`, `FIRESTORE_EMULATOR_HOST` and the client libraries use them
- ⌂ Unit tests mock clients or use emulators

PRODUCTION CHECKLIST

- ☐ Dedicated, least-privilege service account per workload
- ☐ Key creation disabled by org policy
- ☐ No `GOOGLE_APPLICATION_CREDENTIALS` key files on servers

IAM Principals & Roles

principals (users, groups, service accounts) · basic vs predefined vs custom roles · `gcloud projects add-iam-policy-binding` · inheritance down the hierarchy · Google groups for access

WHO CAN DO WHAT

Google Cloud IAM grants **roles** (bundles of permissions) to **principals** (Google accounts, Google groups, service accounts, domains, federated identities) on a resource: organization, folder, project or individual resource. Grants **inherit** downward. **Basic roles** (Owner, Editor, Viewer) are too broad; use **predefined** roles or narrow **custom** roles.

KEY POINTS

- Permissions look like `service.resource.verb` (`run.services.update`)
- Role names: `roles/run.developer`, `roles/storage.objectViewer`
- Grant to Google groups, not individuals
- Allow policies are additive; deny policies override (sheet 06)

GRANT, INSPECT, CUSTOMIZE

```
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --member="group:shop-devs@acme.com" --role="roles/run.developer"
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --member="serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com" \
    --role="roles/cloudsql.client"

# Resource-level grant: one bucket only
gcloud storage buckets add-iam-policy-binding gs://acme-media-prod \
    --member="serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com" \
    --role="roles/storage.objectUser"

gcloud projects get-iam-policy shop-prod-4821 \
    --flatten="bindings[].members" --filter="bindings.members:group:shop-devs" \
    --format="table(bindings.role)"
gcloud iam roles describe roles/run.developer \
    --format="value(includedPermissions)"

gcloud iam roles create shopDeployer --project=shop-prod-4821 \
    --title="Shop Deployer" --stage=GA \
    --permissions=run.services.get,run.services.update,iam.serviceAccounts.actAs
```

CODE EXPLAINED

- → Members are typed: `user:`, `group:`, `serviceAccount:`, `domain:`
- → Resource-level bindings keep access narrow
- → Flatten + filter lists the roles one group holds
- → The custom role has only what a deployer needs

AWS / AZURE EQUIVALENT

- ↔ **AWS** IAM users / roles / policies (aws-05)
- ↔ **Azure** Entra ID + Azure RBAC (az-05, az-06)

COST CHECK

- $ IAM is free
- $ Broad Editor grants let anyone create expensive resources
- $ Use org policies to restrict expensive services or locations

ROLE TYPES

TYPE	EXAMPLE	USE
Basic	`roles/editor`	Avoid in prod
Predefined	`roles/run.developer`	Default choice
Custom	`projects/p/roles/shopDeployer`	Exact needs

PRINCIPAL TYPES

PREFIX	WHO
`user:`	Google account
`group:`	Google group
`serviceAccount:`	Workload identity
`domain:`	Whole Workspace domain
`principal:// / principalSet://`	Federated identities

DO & DON'T

- ✓ **Do** Grant predefined roles to groups at the narrowest resource
- ✓ **Do** Review the IAM recommender's excess-permission suggestions
- ✗ **Don't** Grant `roles/owner` or `roles/editor` to apps
- ✗ **Don't** Grant at the organization when a project would do

LOCAL TESTING

- ⌂ Policy Troubleshooter explains allow / deny for a principal and permission
- ⌂ `gcloud projects get-iam-policy` audits grants
- ⌂ Test with impersonation before granting widely

PRODUCTION CHECKLIST

- □ No basic roles for workloads; groups for humans
- □ IAM recommender reviewed regularly
- □ Bindings managed in Terraform

IAM Policies & Conditions

allow policies & bindings · IAM Conditions · deny policies · Organization Policy constraints · Policy Troubleshooter · role recommender

POLICIES, CONDITIONS AND GUARDRAILS

An **allow policy** on a resource is a list of **bindings**: role + members + optional **condition** (CEL expressions on time, resource name, tags). **Deny policies** block specific permissions regardless of allows. **Organization Policy** constraints restrict configurations (no external IPs, allowed regions, no service account keys) across folders and projects.

KEY POINTS

- Conditions make temporary or scoped access (e.g. expires at a date)
- Deny policies attach at organization, folder or project
- Use etags with `set-iam-policy` to avoid overwriting concurrent changes
- Managed constraints and custom constraints exist for org policy

CONDITION, DENY POLICY, ORG POLICY

```
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --member="user:contractor@acme.com" --role="roles/viewer" \
    --condition-from-file=until-year-end.yaml

ATTACH=cloudresourcemanager.googleapis.com/projects/shop-prod-4821
gcloud iam policies create deny-delete-db --kind=denypolicies \
    --attachment-point=$ATTACH \
    --policy-file=deny.json

gcloud org-policies set-policy no-external-ip.yaml
gcloud org-policies describe compute.vmExternalIpAccess --project=shop-prod-4821

gcloud policy-intelligence troubleshoot-policy iam \
    //cloudresourcemanager.googleapis.com/projects/shop-prod-4821 \
    --principal-email=ada@acme.com --permission=run.services.update
```

CODE EXPLAINED

- → The condition expires access automatically at year end
- → The deny policy blocks instance deletes for everyone (add exceptions as needed)
- → The org policy forbids external IPs on VMs in the project
- → Troubleshooter explains why a permission is allowed or denied

AWS / AZURE EQUIVALENT

- ↔ **AWS** IAM JSON policies, SCPs, permission boundaries (aws-06)
- ↔ **Azure** Azure RBAC, deny assignments, Azure Policy (az-06, az-09)

UNTIL-YEAR-END.YAML / NO-EXTERNAL-IP.YAML

```
# until-year-end.yaml (IAM condition):
expression: request.time < timestamp("2026-12-31T00:00:00Z")
title: until-year-end
---
name: projects/shop-prod-4821/policies/compute.vmExternalIpAccess
spec:
  rules:
    - denyAll: true
# deny.json (deny policy):
# {"rules":[{"denyRule":{"deniedPrincipals":["principalSet://goog/public:all"],
#   "deniedPermissions":["sqladmin.googleapis.com/instances.delete"]}}]}
```

COST CHECK

- $ IAM, deny policies and org policies are free
- $ Org policies like allowed locations prevent costly mistakes
- $ Policy Intelligence tools are free for most uses

LIMITS & QUOTAS

- ≤ 1,500 principals per allow policy (use groups)
- ≤ 300 custom roles per project and per organization
- ≤ Conditions per binding are limited; keep expressions simple

POLICY KINDS

KIND	DOES
`Allow policy`	Grants roles
`Deny policy`	Blocks permissions
`Principal access boundary`	Limits which resources principals can access
`Organization Policy`	Restricts configurations
`VPC Service Controls`	Data exfiltration perimeters

USEFUL CONSTRAINTS

CONSTRAINT	EFFECT
`iam.disableServiceAccountKeyCreation`	No SA keys
`compute.vmExternalIpAccess`	Limit external IPs
`gcp.resourceLocations`	Allowed regions
`storage.publicAccessPrevention`	No public buckets
`iam.allowedPolicyMemberDomains`	Only your domain

DO & DON'T

- ✓ **Do** Use conditions for temporary and scoped access
- ✓ **Do** Set core org policies at the organization
- ✗ **Don't** Use `set-iam-policy` without the current etag
- ✗ **Don't** Rely on reviews alone instead of deny policies for critical actions

LOCAL TESTING

- ⌂ Policy Simulator previews allow-policy changes against recent access
- ⌂ Test org policies in a sandbox folder
- ⌂ Terraform plans show IAM binding diffs

PRODUCTION CHECKLIST

- ☐ Key org policies enforced org-wide
- ☐ Deny policies for destructive admin actions
- ☐ IAM recommender and Policy Analyzer reviews scheduled

Sheet 07

Service Accounts & Workload Identity Federation

service accounts · attaching to resources · `--impersonate-service-account` · Workload Identity Federation for GitHub Actions · Workload Identity Federation for GKE · disabling key creation

IDENTITIES FOR WORKLOADS

A **service account** is the identity a workload runs as. Attach it to Cloud Run, Functions, VMs or GKE workloads and the platform supplies short-lived tokens. People and pipelines **impersonate** service accounts (`roles/iam.serviceAccountTokenCreator`) instead of downloading keys. **Workload Identity Federation (WIF)** lets external identities (GitHub Actions, AWS, Azure, OIDC / SAML) get Google credentials without keys.

SERVICE ACCOUNT AND GITHUB FEDERATION

```
gcloud iam service-accounts create deploy --display-name="GitHub deploy"
SA=deploy@shop-prod-4821.iam.gserviceaccount.com
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --member="serviceAccount:$SA" --role="roles/run.admin"

gcloud iam workload-identity-pools create github --location=global
MAP="google.subject=assertion.sub,attribute.repository=assertion.repository"
COND="assertion.repository=='acme/shop' && assertion.ref=='refs/heads/main'"
gcloud iam workload-identity-pools providers create-oidc gh \
    --location=global --workload-identity-pool=github \
    --issuer-uri=https://token.actions.githubusercontent.com \
    --attribute-mapping="$MAP" --attribute-condition="$COND"

POOL=projects/123456789012/locations/global/workloadIdentityPools/github
REPO=attribute.repository/acme/shop
gcloud iam service-accounts add-iam-policy-binding $SA \
    --role="roles/iam.workloadIdentityUser" \
    --member="principalSet://iam.googleapis.com/$POOL/$REPO"

# People impersonate instead of using keys
gcloud iam service-accounts add-iam-policy-binding $SA \
    --member="group:platform@acme.com" \
    --role="roles/iam.serviceAccountTokenCreator"
gcloud run services list --impersonate-service-account=$SA
```

KEY POINTS

- Default compute service accounts often have Editor: replace with dedicated ones
- Deploying a service with an SA requires `iam.serviceAccounts.actAs` on it
- WIF uses a pool + provider + attribute mapping + condition
- GKE workloads federate through the cluster's workload pool

CODE EXPLAINED

→ The deploy SA gets only the roles CI needs

→ The provider maps GitHub token claims to attributes

→ The condition rejects tokens from other repos or branches

→ `workloadIdentityUser` lets matching GitHub jobs act as the SA

AWS / AZURE EQUIVALENT

↔ **AWS** IAM roles + OIDC providers (aws-07)

↔ **Azure** Managed identities + federated credentials (az-07)

COST CHECK

$ Service accounts, impersonation and WIF are free

$ No key rotation work or leaked-key incidents

$ Each WIF token exchange is a free STS call

SECURITY CHECK

⚠ Pin the WIF attribute condition to your repository (and ref / environment)

⚠ Disable service account key creation by org policy

⚠ Do not use default compute service accounts for workloads

WORKLOAD IDENTITY OPTIONS

WORKLOAD	MECHANISM
Cloud Run / Functions	Attached service account
Compute Engine	Attached SA + scopes
GKE pods	WIF for GKE (principal:// or KSA → SA)
GitHub Actions	WIF OIDC provider
AWS / Azure workloads	WIF AWS / OIDC provider
Humans	Impersonation

ROLES ON SERVICE ACCOUNTS

ROLE	LETS PRINCIPAL
iam.serviceAccountUser	Attach SA to resources (actAs)
iam.serviceAccountTokenCreator	Mint tokens / impersonate
iam.workloadIdentityUser	Federated identities act as SA

DO & DON'T

✓ **Do** One service account per workload with least privilege

✓ **Do** Federate CI and other clouds with WIF

✗ **Don't** Download JSON keys for CI

✗ **Don't** Run apps as the default compute service account

LOCAL TESTING

⌂ Impersonated ADC tests the workload's real permissions

⌂ `gcloud iam service-accounts get-iam-policy` reviews who can act as an SA

⌂ Policy Troubleshooter for permission questions

PRODUCTION CHECKLIST

☐ Zero service account keys (org policy enforced)

☐ WIF conditions pinned to repo + ref / environment

☐ Service account usage monitored (Activity Analysis / recommender)

Secret Manager & Cloud KMS

secrets & versions · `gcloud secrets create / versions access` · rotation · key rings & keys · CMEK · envelope encryption

SECRETS AND ENCRYPTION KEYS

Secret Manager stores secrets as immutable, numbered **versions**; apps read `latest` or a pinned version with IAM (`roles/secretmanager.secretAccessor`). Cloud Run and GKE can mount secrets directly. **Cloud KMS** manages keys in **key rings** per location; services use them as **customer-managed encryption keys (CMEK)**. Data is encrypted at rest by default with Google-managed keys.

KEY POINTS

- Rotation: add a new version, update consumers, disable the old one
- Rotation schedules publish Pub/Sub notifications to trigger your rotator
- Replication policy: automatic or user-managed locations
- Cloud HSM and Cloud EKM add hardware and external key options

SECURITY CHECK

- ⚠ Grant `secretAccessor` per secret, not project-wide
- ⚠ Use CMEK where compliance requires key control and audit
- ⚠ Enable Data Access audit logs for Secret Manager in sensitive projects

LIMITS & QUOTAS

- ≤ Secret payload up to 64 KiB
- ≤ Access requests are quota-limited per project per minute: cache secrets
- ≤ Cloud KMS cryptographic requests have per-project, per-location quotas

SECRETS AND KEYS

```
gcloud secrets create db-password --replication-policy=automatic
printf '%s' "$DB_PASS" | gcloud secrets versions add db-password --data-file=-
gcloud secrets versions access latest --secret=db-password
gcloud secrets add-iam-policy-binding db-password \
    --member="serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com" \
    --role="roles/secretmanager.secretAccessor"
gcloud secrets versions disable 1 --secret=db-password

# Cloud Run reads it as an env var at startup
gcloud run services update api --region=us-central1 \
    --set-secrets=DB_PASSWORD=db-password:latest

gcloud kms keyrings create main --location=us-central1
gcloud kms keys create app --keyring=main --location=us-central1 \
    --purpose=encryption --rotation-period=90d \
    --next-rotation-time=2026-12-01T00:00:00Z
gcloud kms encrypt --key=app --keyring=main --location=us-central1 \
    --plaintext-file=token.txt --ciphertext-file=token.enc
```

CODE EXPLAINED

- → `printf` avoids a trailing newline in the secret
- → The binding grants one service account access to one secret
- → `--set-secrets` injects the secret into the Cloud Run container
- → The key rotates every 90 days automatically

AWS / AZURE EQUIVALENT

- ↔ **AWS** Secrets Manager, Parameter Store, KMS (aws-08)
- ↔ **Azure** Key Vault secrets and keys (az-08)

COST CHECK

- $ Secret Manager: per active secret version per month + per 10,000 accesses
- $ KMS: per key version per month + operations; HSM keys cost more
- $ Disable / destroy old secret versions to stop paying for them

SECRET MANAGER VS KMS

NEED	USE
Passwords, API keys, certs	Secret Manager
Encrypt your data	Cloud KMS (envelope)
Control keys for Google services	CMEK with Cloud KMS
Hardware-backed keys	Cloud HSM
Keys outside Google	Cloud EKM

ROLES

ROLE	GRANTS
secretmanager.secretAccessor	Read values
secretmanager.secretVersionAdder	Add versions
secretmanager.admin	Manage secrets
cloudkms.cryptoKeyEncrypterDecrypter	Use a key
cloudkms.admin	Manage keys

DO & DON'T

- ✓ **Do** Pin versions in production when rollout control matters
- ✓ **Do** Cache secrets in the app for minutes, not per request
- ✗ **Don't** Put secrets in environment variables baked into images
- ✗ **Don't** Destroy KMS key versions still protecting data

LOCAL TESTING

- ⌂ Use a dev project's Secret Manager with impersonated ADC
- ⌂ Abstract secret reads behind an interface for unit tests
- ⌂ gitleaks / secret scanning before pushing

PRODUCTION CHECKLIST

- ☐ Per-secret IAM; access audit logs on
- ☐ Rotation process automated and tested
- ☐ CMEK where required, with key rotation

Security Services

AUDIT, DETECT AND PROTECT

Cloud Audit Logs record admin activity (always on), data access (opt-in), system events and policy denials. **Security Command Center (SCC)** finds misconfigurations, vulnerabilities and threats. **Cloud Armor** is the WAF and DDoS layer for load balancers. **VPC Service Controls** put perimeters around services to stop data exfiltration, **IAP** gates apps and SSH by identity, and **Binary Authorization** only deploys trusted images.

KEY POINTS

- Admin Activity logs are free and kept 400 days
- Data Access logs are off by default for most services (they can be large)
- SCC Standard is free; Premium / Enterprise add threat detection and more
- IAP TCP forwarding replaces bastion hosts

AUDIT, SCC, ARMOR, IAP

```
gcloud logging read 'logName:"cloudaudit.googleapis.com%2Factivity" AND
  protoPayload.methodName:"SetIamPolicy"' --freshness=1d --limit=10 \
  --format="table(timestamp,protoPayload.authenticationInfo.principalEmail)"

gcloud scc findings list organizations/123456789012 \
    --filter='state="ACTIVE" AND severity="HIGH"' --limit=10

gcloud compute security-policies create shop-waf
gcloud compute security-policies rules create 1000 \
    --security-policy=shop-waf \
    --expression="evaluatePreconfiguredWaf('sqli-v33-stable')" \
    --action=deny-403
gcloud compute backend-services update web-be --global \
    --security-policy=shop-waf

gcloud compute ssh web-1 --zone=us-central1-a --tunnel-through-iap
gcloud iap web enable --resource-type=backend-services --service=web-be
```

CODE EXPLAINED

- → Admin activity audit logs show who changed IAM policies
- → SCC lists active high-severity findings for the organization
- → The Cloud Armor rule blocks SQL injection with a preconfigured WAF rule
- → IAP tunnels SSH without external IPs or open port 22 to the world

AWS / AZURE EQUIVALENT

- ↔ **AWS** CloudTrail, GuardDuty, Security Hub, WAF (aws-09)
- ↔ **Azure** Activity Log, Defender for Cloud, Azure Policy, Sentinel (az-09)

COST CHECK

- $ Admin Activity audit logs are free; Data Access logs bill as log ingestion
- $ Cloud Armor: per policy + per rule + per million requests
- $ SCC Premium / Enterprise are subscription priced

SERVICES

SERVICE	DOES
Cloud Audit Logs	Who did what
Security Command Center	Posture + threats
Cloud Armor	WAF, DDoS, rate limits
VPC Service Controls	Data perimeters
IAP	Identity-gated access
Binary Authorization	Trusted images only
Sensitive Data Protection	Find / mask PII

AUDIT LOG TYPES

LOG	DEFAULT
Admin Activity	On, free, 400 days
Data Access	Off (except BigQuery)
System Event	On, free
Policy Denied	On

DO & DON'T

- ✓ **Do** Route audit logs to a central, locked logging project
- ✓ **Do** Put Cloud Armor in front of public load balancers
- ✗ **Don't** Turn on every Data Access log everywhere without a cost plan
- ✗ **Don't** Leave SCC findings without owners

LOCAL TESTING

- ⌂ Prowler and other scanners audit projects from your laptop
- ⌂ Checkov / tfsec scan Terraform before apply
- ⌂ Test VPC SC perimeters in dry-run mode first

PRODUCTION CHECKLIST

- ☐ Org-level log sink for audit logs
- ☐ SCC enabled with findings routed to on-call
- ☐ IAP for admin access; Cloud Armor on public endpoints

VPC Basics

NETWORKING IN GOOGLE CLOUD

A Google Cloud **VPC network** is **global**: one network spans all regions, and **subnets** are regional. Use **custom mode** to choose subnet ranges (auto mode creates a subnet per region). Routes and firewall rules apply network-wide. **Private Google Access** lets VMs without external IPs reach Google APIs, and **Shared VPC** lets many projects use one centrally managed network.

KEY POINTS

- Google reserves 4 addresses in each subnet
- Secondary ranges hold GKE Pod and Service IPs
- Subnets can be expanded later (but not shrunk)
- The `default` network is auto mode: avoid it for production

SECURITY CHECK

- ⚠ Delete or never use the default network in production projects
- ⚠ Turn on Private Google Access for private subnets
- ⚠ Enable VPC Flow Logs on important subnets

LIMITS & QUOTAS

- ≤ VPC networks per project are quota-limited (default 15)
- ≤ Subnet, route and peering counts have per-network limits
- ≤ Check quotas: `gcloud compute project-info describe`

CUSTOM VPC AND SUBNETS

```
gcloud compute networks create vpc-shop --subnet-mode=custom \
    --bgp-routing-mode=global
gcloud compute networks subnets create app-usc1 --network=vpc-shop \
    --region=us-central1 --range=10.10.0.0/20 \
    --enable-private-ip-google-access --enable-flow-logs
gcloud compute networks subnets create gke-usc1 --network=vpc-shop \
    --region=us-central1 --range=10.10.16.0/20 \
    --secondary-range=pods=10.20.0.0/14,services=10.24.0.0/20 \
    --enable-private-ip-google-access
gcloud compute networks subnets create app-euw1 --network=vpc-shop \
    --region=europe-west1 --range=10.11.0.0/20

gcloud compute networks subnets expand-ip-range app-usc1 \
    --region=us-central1 --prefix-length=19
gcloud compute routes list --filter="network:vpc-shop"

# Shared VPC: host project shares the network with service projects
gcloud compute shared-vpc enable net-host-1234
gcloud compute shared-vpc associated-projects add shop-prod-4821 \
    --host-project=net-host-1234
```

CODE EXPLAINED

- → One network holds subnets in two regions
- → Secondary ranges reserve IPs for GKE pods and services
- → Subnets can grow in place
- → Shared VPC centralizes networking for many projects

AWS / AZURE EQUIVALENT

- ↔ **AWS** Regional VPC, per-AZ subnets (aws-10)
- ↔ **Azure** Regional VNet, zone-spanning subnets (az-10)
- ↔ Only Google Cloud makes the network itself global

COST CHECK

- $ VPCs and subnets are free
- $ Egress between regions and to the internet bills per GB
- $ Flow logs bill as log ingestion: sample if large

MODES

MODE	SUBNETS
Auto	One per region, fixed 10.128.0.0/9 ranges
Custom	You choose (production)

KEY FEATURES

FEATURE	USE
Private Google Access	Reach Google APIs without external IP
Secondary ranges	GKE Pods / Services
Shared VPC	Central network, many projects
VPC peering / NCC	Connect networks (sheet 11)
Flow logs	Traffic visibility

DO & DON'T

- ✓ **Do** Plan non-overlapping ranges across regions and environments
- ✓ **Do** Use Shared VPC for organizations with many projects
- ✗ **Don't** Run production in the default network
- ✗ **Don't** Make subnets too small for GKE secondary ranges

LOCAL TESTING

- ⌂ Connectivity Tests explain reachability between endpoints
- ⌂ Plan ranges with `ipcalc` first
- ⌂ Docker networks give a quick local model of isolation

PRODUCTION CHECKLIST

- ☐ Custom-mode VPC in IaC with documented ranges
- ☐ Private Google Access and flow logs enabled
- ☐ Default network deleted (org policy skipDefaultNetworkCreation)

Cloud NAT & Connectivity

OUTBOUND, PRIVATE SERVICES AND HYBRID

Cloud NAT (configured on a **Cloud Router**) gives private VMs and GKE nodes outbound internet access. **Private Service Connect (PSC)** exposes Google APIs or producer services on private IPs in your VPC. **Private services access** peers your VPC with Google-managed services such as Cloud SQL. **Cloud VPN**, **Interconnect** and **Network Connectivity Center** link on-premises and other networks.

KEY POINTS

- Cloud NAT is a managed, software-defined service (no VM to manage)
- Peering is not transitive
- NCC provides hub-and-spoke for VPCs and hybrid links
- Serverless VPC access / Direct VPC egress connect Cloud Run to a VPC

NAT, PSC FOR GOOGLE APIS, PRIVATE SERVICES ACCESS

```
gcloud compute routers create rtr-usc1 --network=vpc-shop --region=us-central1
gcloud compute routers nats create nat-usc1 --router=rtr-usc1 \
    --region=us-central1 --auto-allocate-nat-external-ips \
    --nat-all-subnet-ip-ranges --enable-logging

# Private Service Connect endpoint for all Google APIs
gcloud compute addresses create psc-apis --global \
    --purpose=PRIVATE_SERVICE_CONNECT --addresses=10.255.255.254 \
    --network=vpc-shop
gcloud compute forwarding-rules create pscapis --global --network=vpc-shop \
    --address=psc-apis --target-google-apis-bundle=all-apis

# Private services access (Cloud SQL, Memorystore…)
gcloud compute addresses create google-managed --global \
    --purpose=VPC_PEERING --prefix-length=16 --network=vpc-shop
gcloud services vpc-peerings connect \
    --service=servicenetworking.googleapis.com \
    --ranges=google-managed --network=vpc-shop

gcloud compute networks peerings create shop-to-data --network=vpc-shop \
    --peer-network=vpc-data --peer-project=data-prod-1111
```

CODE EXPLAINED

- → Cloud NAT needs a Cloud Router in the same region
- → The PSC endpoint gives Google APIs a private address in your VPC
- → Private services access reserves a range for Google-managed services
- → Peering needs a matching peering from the other side

AWS / AZURE EQUIVALENT

- ↔ **AWS** NAT gateway, VPC endpoints / PrivateLink, peering, Transit Gateway (aws-11)
- ↔ **Azure** NAT Gateway, Private Endpoints, peering, Virtual WAN (az-11)

COST CHECK

- $ Cloud NAT: per gateway-hour + per GB processed
- $ PSC endpoints: hourly + data processed
- $ Interconnect beats VPN on egress price at high volume

CONNECTIVITY OPTIONS

NEED	USE
Private VMs → internet	Cloud NAT
Private → Google APIs	Private Google Access / PSC
Use Cloud SQL privately	Private services access or PSC
VPC ↔ VPC	Peering / NCC
Office ↔ cloud	HA VPN
Datacenter ↔ cloud	Dedicated / Partner Interconnect

DO & DON'T

- ✓ **Do** Use HA VPN (two tunnels) rather than Classic VPN
- ✓ **Do** Size NAT ports for busy egress (dynamic port allocation)
- ✗ **Don't** Give every VM an external IP to reach the internet
- ✗ **Don't** Forget DNS for PSC endpoints

LOCAL TESTING

- ⌂ Connectivity Tests simulate paths
- ⌂ NAT logs show port exhaustion and drops
- ⌂ Practice in a sandbox VPC with tiny VMs

PRODUCTION CHECKLIST

- ☐ Cloud NAT per region with logging
- ☐ Private access to Google APIs and managed services
- ☐ Hybrid links redundant (HA VPN / dual Interconnect)

Firewall Rules & Policies

FILTERING TRAFFIC

Google Cloud firewalls are stateful and distributed: rules apply at each VM, defined at the network. **VPC firewall rules** have a priority (0–65535, lower wins), direction, action and **targets** (all instances, network tags or service accounts). **Implied rules** deny all ingress and allow all egress. **Hierarchical firewall policies** (org / folder) and **network firewall policies** manage rules centrally.

KEY POINTS

- Target by service account for stronger identity than tags
- Health check ranges: 35.191.0.0/16 and 130.211.0.0/22
- IAP TCP forwarding range: 35.235.240.0/20
- Cloud NGFW adds L7 inspection, FQDN and geo rules

SECURITY CHECK

- ⚠ Never allow 22 / 3389 from 0.0.0.0/0; allow only the IAP range
- ⚠ Use service-account targets instead of free-form tags
- ⚠ Enable firewall logging on deny rules

LIMITS & QUOTAS

- ≤ Firewall rules per project: default quota 100 (adjustable)
- ≤ Network tags: up to 64 per instance
- ≤ Priority range 0–65535 (implied rules at 65535)

RULES FOR A WEB TIER

```
SA_WEB=web@shop-prod-4821.iam.gserviceaccount.com
gcloud compute firewall-rules create allow-lb-health --network=vpc-shop \
    --direction=INGRESS --action=allow --rules=tcp:8080 \
    --source-ranges=35.191.0.0/16,130.211.0.0/22 \
    --target-service-accounts=$SA_WEB --priority=1000
gcloud compute firewall-rules create allow-iap-ssh --network=vpc-shop \
    --direction=INGRESS --action=allow --rules=tcp:22 \
    --source-ranges=35.235.240.0/20 --target-service-accounts=$SA_WEB
gcloud compute firewall-rules create deny-egress-inet --network=vpc-shop \
    --direction=EGRESS --action=deny --rules=all \
    --destination-ranges=0.0.0.0/0 --target-tags=db --priority=2000 \
    --enable-logging

# Network firewall policy managed centrally
gcloud compute network-firewall-policies create shop-fw --global
gcloud compute network-firewall-policies rules create 1000 \
    --firewall-policy=shop-fw --global-firewall-policy \
    --direction=INGRESS --action=allow --layer4-configs=tcp:443 \
    --src-ip-ranges=0.0.0.0/0
gcloud compute network-firewall-policies associations create \
    --firewall-policy=shop-fw --network=vpc-shop --global-firewall-policy
```

CODE EXPLAINED

- → Load balancer health checks come from Google's fixed ranges
- → SSH is allowed only from IAP, never the internet
- → The egress deny keeps database VMs from calling out
- → Network firewall policies group rules and attach to networks

AWS / AZURE EQUIVALENT

↔ **AWS**	Security groups and NACLs (aws-12)
↔ **Azure**	NSGs, ASGs, Azure Firewall (az-12)

COST CHECK

- $ VPC firewall rules are free
- $ Firewall Rules Logging bills as log ingestion
- $ Cloud NGFW Enterprise (L7 inspection) bills per endpoint-hour + GB

RULE FIELDS

FIELD	EXAMPLE
Direction	INGRESS / EGRESS
Priority	1000 (lower wins)
Action	allow / deny
Targets	All, tags, service accounts
Source / destination	CIDRs, tags, SAs
Protocols / ports	tcp:443, udp:53, all

POLICY LEVELS

LEVEL	SCOPE
Hierarchical policy	Organization / folder
Global network policy	One VPC, all regions
Regional network policy	One VPC, one region
VPC firewall rules	Per network (classic)

DO & DON'T

- ✓ **Do** Allow health check and IAP ranges explicitly
- ✓ **Do** Keep org-wide denies in hierarchical policies
- ✗ **Don't** Open admin ports to 0.0.0.0/0
- ✗ **Don't** Rely on tags anyone with instance admin can change

LOCAL TESTING

- ⌂ Connectivity Tests show which rule allowed or blocked traffic
- ⌂ Firewall Insights flags shadowed and unused rules
- ⌂ Test from a VM with `nc -zv` after each change

PRODUCTION CHECKLIST

- ☐ No public admin ports; IAP only
- ☐ Central policies for org-wide rules
- ☐ Firewall logging and Firewall Insights reviewed

Cloud DNS

public & private zones · record sets · DNSSEC · routing policies (weighted, geolocation, failover) · `gcloud dns record-sets` · domain registration through a third-party registrar

MANAGED DNS

Cloud DNS hosts **public zones** on Google's anycast name servers and **private zones** visible only to chosen VPC networks. **Routing policies** answer by weight, geolocation or failover with health checks on internal load balancers. **DNSSEC** can be turned on per public zone. Register domains with any registrar and delegate to the zone's name servers.

KEY POINTS

- Zone DNS names end with a dot: `acme-shop.com.`
- Forwarding zones and server policies support hybrid DNS
- Peering zones resolve names from another VPC
- Response policies override answers inside a VPC

ZONES AND RECORDS

```
gcloud dns managed-zones create shop --dns-name=acme-shop.com. \
    --description="public zone" --dnssec-state=on
gcloud dns managed-zones describe shop --format="value(nameServers)"

gcloud dns record-sets create www.acme-shop.com. --zone=shop \
    --type=A --ttl=300 --rrdatas=34.120.10.20
gcloud dns record-sets create acme-shop.com. --zone=shop --type=TXT \
    --ttl=3600 --rrdatas='"v=spf1 include:_spf.google.com -all"'
gcloud dns record-sets create api.acme-shop.com. --zone=shop --type=A \
    --ttl=60 --routing-policy-type=WRR \
    --routing-policy-data="0.9=34.120.10.21;0.1=34.120.10.22"

# Private zone for internal names
gcloud dns managed-zones create internal --dns-name=shop.internal. \
    --description="private" --visibility=private --networks=vpc-shop
gcloud dns record-sets create db.shop.internal. --zone=internal \
    --type=A --ttl=300 --rrdatas=10.10.0.5
gcloud dns record-sets list --zone=shop
```

CODE EXPLAINED

- → Copy the name servers to your registrar to delegate
- → DNSSEC on means you must also add the DS record at the registrar
- → The weighted policy sends 90% / 10% of answers
- → Private zones resolve only inside `vpc-shop`

AWS / AZURE EQUIVALENT

- ↔ **AWS** Route 53 hosted zones and routing policies (aws-13)
- ↔ **Azure** Azure DNS + Traffic Manager (az-13)

COST CHECK

- $ Per managed zone per month + per million queries
- $ Private zones bill like public ones
- $ Health-checked routing policies add health check costs

ROUTING POLICIES

POLICY	ROUTES BY
WRR	Weights
Geolocation	Client location
Failover (primary / backup)	Health of internal LB targets
Default	Plain records

ZONE TYPES

TYPE	USE
Public	Internet names
Private	Names inside VPCs
Forwarding	Send queries to on-prem DNS
Peering	Use another VPC's DNS
Response policy	Override answers

DO & DON'T

- ✓ **Do** Manage zones and records in Terraform
- ✓ **Do** Add CAA records for your certificate authority
- ✗ **Don't** Forget the DS record when enabling DNSSEC
- ✗ **Don't** Use long TTLs on records you may change in an incident

LOCAL TESTING

- ⌂ `dig @ns-cloud-a1.googledomains.com www.acme-shop.com` tests before delegation
- ⌂ `dig +dnssec` checks signatures
- ⌂ CoreDNS in Docker for local name experiments

PRODUCTION CHECKLIST

- ☐ Records in IaC; DNSSEC with DS at registrar
- ☐ Private zones for internal service names
- ☐ Short TTLs on failover records

Cloud CDN & Certificates

Cloud CDN on an external Application Load Balancer · backend buckets · cache modes & invalidation · Certificate Manager · Google-managed certificates · Media CDN pointer

EDGE CACHING AND TLS

Cloud CDN caches responses at Google's edge for **external Application Load Balancers**, from **backend buckets** (Cloud Storage) or backend services (VMs, GKE, Cloud Run via serverless NEGs). **Cache modes** decide what is cached. **Certificate Manager** issues and renews **Google-managed certificates** (DNS authorization works before cut-over) and maps them to load balancers.

KEY POINTS

- CACHE_ALL_STATIC caches common static types automatically
- Signed URLs / cookies protect private cached content
- Invalidations are rate-limited: prefer versioned file names
- Media CDN targets large video and download workloads

SECURITY CHECK

- ⚠ Keep the bucket private; serve through the load balancer
- ⚠ Use an SSL policy with TLS 1.2+ (MODERN or RESTRICTED profile)
- ⚠ Attach Cloud Armor to the backend

STATIC SITE BEHIND A GLOBAL ALB WITH CDN

```
gcloud compute backend-buckets create site-be --gcs-bucket-name=acme-site \
    --enable-cdn --cache-mode=CACHE_ALL_STATIC --default-ttl=3600
gcloud compute url-maps create site-map --default-backend-bucket=site-be

gcloud certificate-manager dns-authorizations create shop-auth \
    --domain=shop.acme.com
gcloud certificate-manager certificates create shop-cert \
    --domains=shop.acme.com --dns-authorizations=shop-auth
gcloud certificate-manager maps create shop-map
gcloud certificate-manager maps entries create shop-entry --map=shop-map \
    --certificates=shop-cert --hostname=shop.acme.com

gcloud compute ssl-policies create modern-tls --profile=MODERN \
    --min-tls-version=1.2
gcloud compute target-https-proxies create site-proxy --url-map=site-map \
    --certificate-map=shop-map --ssl-policy=modern-tls
gcloud compute addresses create site-ip --global
gcloud compute forwarding-rules create site-https --global \
    --target-https-proxy=site-proxy --address=site-ip --ports=443

gcloud compute url-maps invalidate-cdn-cache site-map --path="/index.html"
```

CODE EXPLAINED

- → The backend bucket serves the Cloud Storage site with CDN on
- → DNS authorization lets the certificate issue before DNS points at the LB
- → The certificate map attaches certificates by hostname
- → The forwarding rule exposes one global anycast IP on 443

AWS / AZURE EQUIVALENT

- ↔ **AWS** CloudFront + ACM (aws-14)
- ↔ **Azure** Front Door + managed certificates (az-14)

COST CHECK

- $ Cache egress and cache fill per GB, plus lookups
- $ Forwarding rules bill hourly
- $ Google-managed certificates through Certificate Manager have a small per-certificate fee beyond a free tier

CACHE MODES

MODE	CACHES
CACHE_ALL_STATIC	Static types + Cache-Control
USE_ORIGIN_HEADERS	Only what origin marks cacheable
FORCE_CACHE_ALL	Everything (careful with private data)

ALB PIECES

PIECE	ROLE
Forwarding rule	IP + port
Target HTTPS proxy	TLS, certificates
URL map	Host / path routing
Backend service / bucket	Where traffic goes
Health check	Backend health

DO & DON'T

- ✓ **Do** Version asset file names; invalidate only entry points
- ✓ **Do** Use Certificate Manager with DNS authorization for zero-downtime cut-overs
- ✗ **Don't** Make the site bucket public
- ✗ **Don't** Use FORCE_CACHE_ALL on pages with user data

LOCAL TESTING

- ⌂ Preview the site locally with `npx serve`
- ⌂ `curl -I` shows `age` and cache headers after deploy
- ⌂ Test URL maps with `gcloud compute url-maps validate`

PRODUCTION CHECKLIST

- ☐ Private bucket behind the ALB
- ☐ Cloud Armor and modern SSL policy attached
- ☐ HTTP → HTTPS redirect URL map

Compute Engine Basics

machine families (E2, N4, C4, T2D…) · images & boot disks · `gcloud compute instances create` · startup scripts & metadata · OS Login · Shielded VM

VIRTUAL MACHINES

Compute Engine runs VMs in a **zone** from a public or custom **image**. Pick a **machine type** from a family (general purpose E2 / N4 / C4, compute-optimized, memory-optimized, Arm-based C4A / T2A), attach Persistent or Hyperdisk volumes, a service account and network tags. **Metadata** carries startup scripts and config; **OS Login** ties SSH access to IAM.

KEY POINTS

- Custom machine types choose exact vCPU and memory on many families
- Startup scripts run as root on every boot
- Spot VMs are cheap but can be preempted any time
- Shielded VM (Secure Boot, vTPM, integrity monitoring) is on by default for supported images

SECURITY CHECK

- ⚠ No external IP: use IAP for SSH and Cloud NAT for egress
- ⚠ Attach a dedicated service account (not the default compute SA)
- ⚠ Enable OS Login and block project-wide SSH keys

LIMITS & QUOTAS

- ≤ vCPU quotas per region and per family (often small on new projects)
- ≤ External IP address quota per region
- ≤ GPU quotas start at 0: request ahead

CREATE AND CONNECT

```
gcloud compute instances create web-1 --zone=us-central1-a \
    --machine-type=e2-standard-2 \
    --image-family=debian-12 --image-project=debian-cloud \
    --boot-disk-size=20GB --boot-disk-type=pd-balanced \
    --subnet=app-usc1 --no-address \
    --service-account=web@shop-prod-4821.iam.gserviceaccount.com \
    --scopes=cloud-platform --tags=web --labels=env=prod,app=shop \
    --metadata=enable-oslogin=TRUE,block-project-ssh-keys=TRUE \
    --metadata-from-file=startup-script=startup.sh \
    --shielded-secure-boot

gcloud compute ssh web-1 --zone=us-central1-a --tunnel-through-iap
gcloud compute instances list --filter="labels.app=shop" \
    --format="table(name,zone.basename(),status,networkInterfaces[0].networkIP)"
gcloud compute instances stop web-1 --zone=us-central1-a
gcloud compute instances set-machine-type web-1 --zone=us-central1-a \
    --machine-type=e2-standard-4
gcloud compute instances start web-1 --zone=us-central1-a
```

CODE EXPLAINED

- → `--no-address` keeps the VM private
- → `cloud-platform` scope + IAM roles is the modern pattern
- → OS Login grants SSH via IAM roles (`roles/compute.osLogin`)
- → Machine type changes need a stopped VM

AWS / AZURE EQUIVALENT

↔	**AWS**	EC2 instances, AMIs, user data (aws-15)
↔	**Azure**	Azure VMs, images, cloud-init (az-15)

STARTUP.SH

```
#!/bin/bash
apt-get update && apt-get install -y nginx
systemctl enable --now nginx
```

COST CHECK

- $ Billed per second (1-minute minimum) for vCPU and memory
- $ Stopped VMs still pay for disks and reserved IPs
- $ Sustained use discounts apply automatically to N1 / N2 and some other families

MACHINE FAMILIES

FAMILY	USE
E2	Cost-optimized general purpose
N4 / N2 / N2D	Balanced general purpose
C4 / C4A / C3	High performance (C4A is Arm)
T2D / T2A	Scale-out (T2A is Arm)
M3 / X4	Memory-optimized
A3 / A4 / G2	GPUs

DISK TYPES

TYPE	USE
pd-balanced	Default SSD-backed
pd-ssd	Higher IOPS
pd-standard	Cheap HDD
Hyperdisk Balanced / Extreme	Tunable IOPS / throughput
Local SSD	Ephemeral scratch

DO & DON'T

- ✓ **Do** Use instance templates and MIGs for anything that should scale
- ✓ **Do** Prefer image families for current OS images
- ✗ **Don't** Give VMs external IPs by default
- ✗ **Don't** Use the default compute service account with Editor

LOCAL TESTING

- ⌂ Test startup scripts in a local VM or container first
- ⌂ `gcloud compute instances get-serial-port-output` debugs boot
- ⌂ Packer builds custom images reproducibly

PRODUCTION CHECKLIST

- ☐ Private VMs, IAP SSH, OS Login
- ☐ Dedicated SA with least privilege
- ☐ Built from a template in IaC, not by hand

Sheet 16

Compute Engine Operations & Pricing

Spot VMs · committed use discounts · sustained use discounts · snapshots & machine images · live migration · VM Manager (OS patching)

RUNNING VMS CHEAPLY AND SAFELY

Compute Engine pricing has three levers: **sustained use discounts** (automatic on some families), **committed use discounts** (1 or 3 years, resource- or spend-based) and **Spot VMs** (deep discount, preemptible). **Snapshots** back up disks; **machine images** capture a whole VM. **Live migration** moves VMs during host maintenance, and **VM Manager** patches and inventories OSes.

KEY POINTS

- Spot VMs get a 30-second shutdown notice
- Snapshot schedules automate disk backups
- Flexible CUDs apply across families and regions
- Rightsizing recommendations come from the Recommender

SPOT, SNAPSHOTS, COMMITMENTS, PATCHING

```
gcloud compute instances create batch-1 --zone=us-central1-a \
    --machine-type=c4-standard-8 --provisioning-model=SPOT \
    --instance-termination-action=DELETE \
    --image-family=debian-12 --image-project=debian-cloud --no-address

gcloud compute resource-policies create snapshot-schedule daily-7 \
    --region=us-central1 --daily-schedule --start-time=03:00 \
    --max-retention-days=7
gcloud compute disks add-resource-policies web-1 --zone=us-central1-a \
    --resource-policies=daily-7
gcloud compute snapshots create web-1-pre-upgrade \
    --source-disk=web-1 --source-disk-zone=us-central1-a
gcloud compute machine-images create web-1-img --source-instance=web-1 \
    --source-instance-zone=us-central1-a

gcloud compute commitments create cud-web --region=us-central1 \
    --plan=36-month --resources=vcpu=16,memory=64GB

gcloud compute os-config patch-jobs execute --instance-filter-all \
    --display-name=monthly --duration=3600s
gcloud recommender recommendations list --location=us-central1-a \
    --recommender=google.compute.instance.MachineTypeRecommender
```

CODE EXPLAINED

- → The Spot VM is deleted when Google reclaims capacity
- → The resource policy snapshots the disk daily and keeps 7 days
- → Machine images capture all disks plus configuration
- → The recommender suggests smaller machine types for idle VMs

AWS / AZURE EQUIVALENT

- ↔ **AWS** Spot, Savings Plans, EBS snapshots, SSM Patch Manager (aws-16)
- ↔ **Azure** Spot VMs, reservations, Azure Backup, Update Manager (az-16)

COST CHECK

- $ Spot VMs: up to 60–91% off on-demand
- $ CUDs: up to 55% off (more for memory-optimized) for 1–3 years
- $ Snapshots bill per GB stored (incremental)

DISCOUNTS

OPTION	COMMITMENT	BEST FOR
Sustained use	None (automatic)	Steady VMs on N1/N2
Resource-based CUD	1 / 3 years	Known vCPU / memory
Flexible CUD	1 / 3 years spend	Changing families
Spot VM	None, preemptible	Batch, CI, stateless

BACKUPS

TOOL	CAPTURES
Snapshot	One disk, incremental
Snapshot schedule	Automated snapshots
Machine image	Whole VM
Backup and DR Service	Policies, vaults, apps

DO & DON'T

- ✓ **Do** Run batch and CI on Spot VMs
- ✓ **Do** Buy CUDs only for the steady baseline
- ✗ **Don't** Forget snapshot retention (they pile up)
- ✗ **Don't** Treat live migration as a substitute for multi-zone design

LOCAL TESTING

- ⌂ Simulate preemption: `gcloud compute instances simulate-maintenance-event`
- ⌂ Restore a snapshot to a scratch VM to test backups
- ⌂ Pricing calculator for CUD vs on-demand

PRODUCTION CHECKLIST

- ☐ Snapshot schedules on every persistent disk
- ☐ CUDs covering baseline; Spot for batch
- ☐ VM Manager patch policies with reporting

Cloud Load Balancing

Application vs Network Load Balancers · external vs internal · global vs regional · backend services & NEGs · health checks · serverless NEGs for Cloud Run

CHOOSING A LOAD BALANCER

Cloud Load Balancing is a family: **Application Load Balancers** (HTTP/S, L7) and **Network Load Balancers** (TCP/UDP, L4, proxy or passthrough), each **external** or **internal**, **global** or **regional**. They share building blocks: a **forwarding rule** (IP + port), a **target proxy**, a **URL map**, **backend services** with **health checks**, and backends such as MIGs or **NEGs** (including serverless NEGs for Cloud Run).

KEY POINTS

- Global external ALB gives one anycast IP worldwide
- Health check traffic comes from 35.191.0.0/16 and 130.211.0.0/22
- Serverless NEGs front Cloud Run, Functions and App Engine
- Cloud Armor and Cloud CDN attach to backend services

GLOBAL EXTERNAL ALB FOR A MIG AND CLOUD RUN

```
gcloud compute health-checks create http hc-web --port=8080 \
    --request-path=/healthz
gcloud compute backend-services create web-be --global \
    --protocol=HTTP --port-name=http --health-checks=hc-web \
    --load-balancing-scheme=EXTERNAL_MANAGED
gcloud compute backend-services add-backend web-be --global \
    --instance-group=web-mig --instance-group-region=us-central1

gcloud compute network-endpoint-groups create api-neg \
    --region=us-central1 --network-endpoint-type=serverless \
    --cloud-run-service=api
gcloud compute backend-services create api-be --global \
    --load-balancing-scheme=EXTERNAL_MANAGED
gcloud compute backend-services add-backend api-be --global \
    --network-endpoint-group=api-neg --network-endpoint-group-region=us-central1

gcloud compute url-maps create shop-map --default-service=web-be
gcloud compute url-maps add-path-matcher shop-map --path-matcher-name=api \
    --default-service=web-be --path-rules="/api/*=api-be" \
    --new-hosts=shop.acme.com
gcloud compute target-https-proxies create shop-proxy --url-map=shop-map \
    --certificate-map=shop-map
gcloud compute forwarding-rules create shop-https --global \
    --load-balancing-scheme=EXTERNAL_MANAGED \
    --target-https-proxy=shop-proxy --ports=443
```

CODE EXPLAINED

- → The health check probes `/healthz` on port 8080
- → The MIG backend serves the website
- → The serverless NEG routes `/api/*` to Cloud Run
- → The forwarding rule exposes the global IP on 443

AWS / AZURE EQUIVALENT

- ↔ **AWS** ALB, NLB, target groups (aws-17)
- ↔ **Azure** Application Gateway, Load Balancer, Front Door (az-17)

COST CHECK

- $ Forwarding rules bill hourly (first 5 rules priced together)
- $ Data processed by the LB bills per GB
- $ Global ALB + CDN can cut egress cost for cacheable content

WHICH LOAD BALANCER

NEED	USE
Public website / API, global	Global external ALB
Public, one region	Regional external ALB
Internal HTTP between services	Internal ALB
TCP/UDP, preserve client IP	Passthrough NLB
TCP proxy, TLS offload	Proxy NLB
Front Cloud Run	ALB + serverless NEG

PIECES

PIECE	ROLE
Forwarding rule	IP, port, scheme
Target proxy	HTTP(S) / TCP / SSL
URL map	Host / path rules
Backend service	Balancing, health, Armor, CDN
Backend	MIG, NEG, bucket

DO & DON'T

- ✓ **Do** Use EXTERNAL_MANAGED (the modern ALB scheme)
- ✓ **Do** Point health checks at a cheap `/healthz` endpoint
- ✗ **Don't** Forget firewall rules for health check ranges
- ✗ **Don't** Expose Cloud Run publicly when it should only take LB traffic (set ingress)

LOCAL TESTING

- ⌂ Test URL maps with `gcloud compute url-maps validate`
- ⌂ Check backend health: `gcloud compute backend-services get-health`
- ⌂ Run the app behind a local reverse proxy (Caddy / nginx) to test paths

PRODUCTION CHECKLIST

- ☐ HTTPS only with a modern SSL policy
- ☐ Cloud Armor attached to public backends
- ☐ Health checks, logging and alerting on 5xx

Managed Instance Groups

SELF-HEALING, SCALING VMS

A **managed instance group (MIG)** runs identical VMs from an **instance template**. **Regional MIGs** spread across zones. The **autoscaler** adds and removes VMs on CPU, load balancer utilization, Cloud Monitoring metrics or schedules. **Autohealing** recreates VMs that fail a health check, and the **updater** rolls out new templates with surge, max-unavailable and canary versions.

KEY POINTS

- Instance templates are immutable: create a new one to change
- Initial delay prevents autohealing during boot
- Predictive autoscaling forecasts daily load patterns
- Stateful MIGs keep disks and IPs for stateful apps

TEMPLATE, REGIONAL MIG, AUTOSCALING, ROLLOUT

```
gcloud compute instance-templates create web-v2 --region=us-central1 \
    --machine-type=e2-standard-2 --subnet=app-usc1 --no-address \
    --image-family=debian-12 --image-project=debian-cloud \
    --service-account=web@shop-prod-4821.iam.gserviceaccount.com \
    --scopes=cloud-platform --metadata-from-file=startup-script=startup.sh

gcloud compute instance-groups managed create web-mig --region=us-central1 \
    --template=web-v1 --size=3 --health-check=hc-web --initial-delay=120
gcloud compute instance-groups managed set-named-ports web-mig \
    --region=us-central1 --named-ports=http:8080
gcloud compute instance-groups managed set-autoscaling web-mig \
    --region=us-central1 --min-num-replicas=3 --max-num-replicas=12 \
    --target-cpu-utilization=0.6 --cool-down-period=90

# Canary 10% on v2, then roll everyone
gcloud compute instance-groups managed rolling-action start-update web-mig \
    --region=us-central1 --version=template=web-v1 \
    --canary-version=template=web-v2,target=10% --max-surge=3
gcloud compute instance-groups managed rolling-action start-update web-mig \
    --region=us-central1 --version=template=web-v2 \
    --max-surge=3 --max-unavailable=0
gcloud compute instance-groups managed list-instances web-mig \
    --region=us-central1
```

CODE EXPLAINED

- → The template is the blueprint for every VM
- → Autohealing waits 120 s before judging new VMs
- → The autoscaler keeps CPU near 60% between 3 and 12 VMs
- → The canary puts 10% on v2 before the full rollout

AWS / AZURE EQUIVALENT

↔	**AWS**	Launch templates + Auto Scaling groups (aws-18)
↔	**Azure**	Virtual Machine Scale Sets (az-18)

COST CHECK

- $ MIGs and the autoscaler are free; you pay for VMs
- $ Mix Spot VMs in a separate MIG for cheap burst capacity
- $ Scale-in settings stop flapping that wastes boot time

LIMITS & QUOTAS

- ≤ Up to 2,000 instances per regional MIG (1,000 zonal)
- ≤ Autoscaler needs a cool-down / initialization period
- ≤ Regional vCPU quotas still cap scale-out

AUTOSCALING SIGNALS

SIGNAL	EXAMPLE
CPU	target 0.6
LB utilization	backend capacity %
Monitoring metric	Pub/Sub backlog per VM
Schedule	Weekday business hours
Predictive	Forecast daily patterns

UPDATE SETTINGS

SETTING	MEANING
--max-surge	Extra VMs during update
--max-unavailable	VMs down at once
--canary-version	Second template at %
--replacement-method	substitute / recreate
--minimal-action	refresh / restart / replace

DO & DON'T

- ✓ **Do** Use regional MIGs for production
- ✓ **Do** Bake apps into images for fast scale-out
- ✗ **Don't** Patch VMs in place inside a MIG
- ✗ **Don't** Set the autohealing initial delay shorter than boot time

LOCAL TESTING

- ⌂ Test the startup script on a single VM first
- ⌂ Watch updates: `gcloud compute instance-groups managed describe`
- ⌂ Load test with k6 / hey to see autoscaling react

PRODUCTION CHECKLIST

- ☐ Regional MIG with health check + autohealing
- ☐ Canary rollouts for new templates
- ☐ Autoscaler bounds set within quota

Choosing Compute

Cloud Run vs GKE vs Compute Engine · Cloud Run functions · App Engine (legacy choice) · Batch · GKE Autopilot vs Standard · decision table

WHICH COMPUTE SERVICE

Default to **Cloud Run** for containers and HTTP or event-driven apps: it scales to zero and needs no cluster. Use **Cloud Run functions** for small event handlers, **GKE** (Autopilot first) when you need Kubernetes, **Compute Engine** for full OS control or licensed software, and **Batch** for job queues on VMs. App Engine still runs but new apps usually choose Cloud Run.

KEY POINTS

- Cloud Run functions are Cloud Functions now built on Cloud Run
- GKE Autopilot bills per Pod resource requests
- Cloud Run supports GPUs for inference
- Batch schedules jobs on Compute Engine VMs (incl. Spot)

SAME APP, THREE WAYS

```
# Cloud Run: container → URL
gcloud run deploy api --source=. --region=us-central1 \
    --service-account=api@shop-prod-4821.iam.gserviceaccount.com

# GKE Autopilot: cluster + Kubernetes manifests
gcloud container clusters create-auto shop --region=us-central1
kubectl create deployment api \
    --image=us-central1-docker.pkg.dev/shop-prod-4821/apps/api:1.4.2
kubectl expose deployment api --port=80 --target-port=8080

# Compute Engine: VM running the container directly
gcloud compute instances create api-vm --zone=us-central1-a \
    --machine-type=e2-small --image-family=cos-stable \
    --image-project=cos-cloud --no-address \
    --metadata-from-file=user-data=cloud-init.yaml

# Batch: run 100 tasks on Spot VMs
gcloud batch jobs submit resize-images --location=us-central1 \
    --config=batch-job.json
```

CODE EXPLAINED

- → `--source` builds the container with Cloud Build
- → Autopilot manages nodes; you manage workloads
- → Container-Optimized OS runs the container from cloud-init
- → Batch handles queuing, retries and VM lifecycle

AWS / AZURE EQUIVALENT

- ↔ **AWS** Lambda, Fargate, EKS, EC2, Batch (aws-19)
- ↔ **Azure** Functions, Container Apps, AKS, VMs (az-19)

COST CHECK

- $ Cloud Run: per vCPU-second, GiB-second and requests; scales to zero
- $ GKE: cluster management fee (free tier covers one zonal / Autopilot cluster) + nodes or Pods
- $ VMs bill while running, idle or not

PICK A SERVICE

NEED	USE
HTTP API / web app container	Cloud Run
Small event handler	Cloud Run functions
Run-to-completion container	Cloud Run jobs
Kubernetes ecosystem	GKE Autopilot
Full node control	GKE Standard
OS control / licensed apps	Compute Engine
Large batch on VMs	Batch

TRADE-OFFS

SERVICE	YOU MANAGE
Cloud Run	Container + config
GKE Autopilot	Workloads (Kubernetes)
GKE Standard	Workloads + node pools
Compute Engine	OS, runtime, scaling

DO & DON'T

- ✓ **Do** Start with Cloud Run; move only when you hit a real limit
- ✓ **Do** Containerize so you can switch platforms later
- ✗ **Don't** Run a GKE cluster for one small app
- ✗ **Don't** Run idle VMs for spiky workloads

LOCAL TESTING

- ⌂ Same container runs locally with `docker run`
- ⌂ Cloud Code (VS Code / JetBrains) deploys to Cloud Run and GKE
- ⌂ kind / minikube for local Kubernetes

PRODUCTION CHECKLIST

- ☐ Choice documented with reasons (ADR)
- ☐ Container images built once, promoted across environments
- ☐ Cost reviewed monthly per platform

Cloud Run Basics

SERVERLESS CONTAINERS

Cloud Run runs any container that listens on $PORT. Each deploy creates an immutable **revision**; **traffic** can be split between revisions for canaries. Instances scale from zero (or a **min instances** floor) up to **max instances**, each handling up to **concurrency** requests at once. **Ingress** and IAM **authentication** decide who can call it.

KEY POINTS

- Default concurrency is 80 requests per instance
- Request-based billing charges CPU only during requests; instance-based keeps CPU always allocated
- Revisions can be tagged for test URLs
- Direct VPC egress connects to private VPC resources

SECURITY CHECK

- ⚠ Use `--no-allow-unauthenticated` unless the service is public
- ⚠ Set ingress to `internal-and-cloud-load-balancing` behind an ALB
- ⚠ Run as a dedicated service account, not the default compute SA

LIMITS & QUOTAS

- ≤ Request timeout up to 60 minutes (default 5)
- ≤ Max concurrency 1,000 per instance
- ≤ Memory up to 32 GiB, CPU up to 8 vCPU per instance; max instances limited by regional quota

DEPLOY, CONFIGURE, SPLIT TRAFFIC

```
REPO=us-central1-docker.pkg.dev/shop-prod-4821/apps
gcloud run deploy api --image=$REPO/api:1.4.2 --region=us-central1 \
    --service-account=api@shop-prod-4821.iam.gserviceaccount.com \
    --no-allow-unauthenticated --ingress=internal-and-cloud-load-balancing \
    --cpu=1 --memory=512Mi --concurrency=80 --timeout=60s \
    --min-instances=1 --max-instances=50 \
    --set-env-vars=APP_ENV=prod --set-secrets=DB_PASSWORD=db-password:latest \
    --network=vpc-shop --subnet=app-usc1 --vpc-egress=private-ranges-only

# Canary: deploy without traffic, test the tag URL, then shift
gcloud run deploy api --image=$REPO/api:1.5.0 --region=us-central1 \
    --no-traffic --tag=canary
gcloud run services update-traffic api --region=us-central1 \
    --to-tags=canary=10
gcloud run services update-traffic api --region=us-central1 --to-latest

gcloud run services add-iam-policy-binding api --region=us-central1 \
    --member="serviceAccount:web@shop-prod-4821.iam.gserviceaccount.com" \
    --role="roles/run.invoker"
gcloud run revisions list --service=api --region=us-central1
```

CODE EXPLAINED

- → Only the load balancer and internal traffic reach the service
- → Min 1 instance avoids cold starts for the first request
- → `--no-traffic --tag` creates a test URL for the new revision
- → `run.invoker` lets the web service call the API

AWS / AZURE EQUIVALENT

- ↔ **AWS** Lambda / App Runner / ECS Fargate (aws-20)
- ↔ **Azure** Azure Container Apps (az-20)

COST CHECK

- $ Per vCPU-second, GiB-second and per million requests, with a monthly free tier
- $ Min instances bill while idle (at a lower idle rate)
- $ Higher concurrency means fewer instances and lower cost

KEY SETTINGS

FLAG	CONTROLS
`--concurrency`	Requests per instance
`--min / --max-instances`	Scaling floor / ceiling
`--cpu-boost`	Faster cold starts
`--no-cpu-throttling`	Instance-based billing
`--ingress`	all / internal / internal-and-cloud-load-balancing
`--execution-environment`	gen1 / gen2

WHO CAN CALL

SETTING	EFFECT
`--allow-unauthenticated`	Public (allUsers invoker)
`--no-allow-unauthenticated`	IAM ID token required
`IAP on the service`	User sign-in
`Ingress internal`	VPC / same-project sources only

DO & DON'T

- ✓ **Do** Keep containers stateless and fast to start
- ✓ **Do** Use tags + traffic splitting for canaries
- ✗ **Don't** Do background work after returning a response with request-based billing
- ✗ **Don't** Make internal APIs public

LOCAL TESTING

- ⌂ `docker run -p 8080:8080 -e PORT=8080 api`
- ⌂ `gcloud run services proxy api` tunnels to a private service with your identity
- ⌂ Cloud Code runs a local Cloud Run emulator

PRODUCTION CHECKLIST

- ☐ Dedicated SA, private ingress, IAM auth
- ☐ Min instances for latency-sensitive services
- ☐ Revisions deployed by CI with canary + rollback

Cloud Run Functions & Jobs

Cloud Run functions (formerly Cloud Functions) · HTTP & event triggers · Functions Framework · Cloud Run jobs · tasks & parallelism · scheduling jobs

FUNCTIONS AND RUN-TO-COMPLETION WORK

Cloud Run functions deploy source code (Node.js, Python, Go, Java, .NET, Ruby, PHP) as a Cloud Run service built with the **Functions Framework**, triggered by HTTP or **Eventarc** events. **Cloud Run jobs** run containers to completion: a job execution starts many **tasks** in parallel, each with an index, retries and a timeout. Cloud Scheduler can run jobs on a cron.

KEY POINTS

- `gcloud functions deploy --gen2` and `gcloud run deploy --function` both create Cloud Run functions
- Tasks read `CLOUD_RUN_TASK_INDEX` and `CLOUD_RUN_TASK_COUNT`
- Event functions receive CloudEvents
- Jobs can run up to 168 hours per task (7 days)

FUNCTION CODE (PYTHON)

```python
import functions_framework

@functions_framework.http
def hello(request):
    name = request.args.get("name", "world")
    return {"hello": name}

@functions_framework.cloud_event
def on_upload(event):
    data = event.data
    print(f"New object gs://{data['bucket']}/{data['name']}")
```

CODE EXPLAINED

- → One decorator per trigger type
- → `--trigger-bucket` wires an Eventarc trigger for new objects
- → The job runs 50 tasks, 10 at a time, retrying failures
- → Cloud Scheduler calls the Run Admin API to start the job nightly

AWS / AZURE EQUIVALENT

- ↔ **AWS** Lambda functions, ECS tasks / Batch (aws-21)
- ↔ **Azure** Azure Functions, Container Apps jobs (az-21)

DEPLOY FUNCTION, JOB, SCHEDULE

```
gcloud run deploy hello --source=. --function=hello \
    --base-image=python312 --region=us-central1 --no-allow-unauthenticated

gcloud functions deploy on-upload --gen2 --runtime=python312 \
    --region=us-central1 --source=. --entry-point=on_upload \
    --trigger-bucket=acme-uploads-prod \
    --service-account=fn@shop-prod-4821.iam.gserviceaccount.com

REPO=us-central1-docker.pkg.dev/shop-prod-4821/apps
gcloud run jobs create resize --image=$REPO/resize:1.0 \
    --region=us-central1 --tasks=50 --parallelism=10 \
    --max-retries=3 --task-timeout=15m
gcloud run jobs execute resize --region=us-central1 --wait

RUN=https://run.googleapis.com/v2
gcloud scheduler jobs create http resize-nightly --location=us-central1 \
    --schedule="0 2 * * *" --http-method=POST \
    --uri="$RUN/projects/shop-prod-4821/locations/us-central1/jobs/resize:run" \
    --oauth-service-account-email=sched@shop-prod-4821.iam.gserviceaccount.com
```

COST CHECK

- $ Functions bill as Cloud Run (CPU, memory, requests)
- $ Jobs bill only while tasks run
- $ Scheduler: a few free jobs per billing account, then per job per month

LIMITS & QUOTAS

- ≤ Up to 10,000 tasks per job execution
- ≤ Task timeout default 10 min, max 168 hours
- ≤ Event-driven functions inherit Eventarc delivery and retry limits

TRIGGERS

TRIGGER	SOURCE
HTTP	Direct request
Cloud Storage	Object finalized / deleted
Pub/Sub	Message published
Eventarc (audit logs)	Almost any Google API call
Firestore	Document changes
Scheduler	Cron (HTTP / Pub/Sub)

JOB SETTINGS

FLAG	MEANING
--tasks	Total tasks
--parallelism	Run at once
--max-retries	Per task
--task-timeout	Per task limit
--args / --set-env-vars	Per execution overrides

DO & DON'T

- ✓ **Do** Make event handlers idempotent (events can repeat)
- ✓ **Do** Split big batches by task index
- ✗ **Don't** Use a function for long-running batch work (use jobs)
- ✗ **Don't** Process an upload event by writing back to the same bucket path (loop)

LOCAL TESTING

- ⌂ `functions-framework --target=hello` runs locally on port 8080
- ⌂ Send test CloudEvents with curl to the local server
- ⌂ Run job containers locally with `CLOUD_RUN_TASK_INDEX=0`

PRODUCTION CHECKLIST

- ☐ Dedicated SA per function / job
- ☐ Dead-letter topics on event triggers
- ☐ Alerts on job failures and function errors

API Gateway & Apigee

PUBLISHING APIS

API Gateway fronts Cloud Run, Functions and App Engine with an **OpenAPI 2.0** config: auth (API keys, Google ID tokens, JWT from any issuer), quotas and routing, fully managed. **Apigee** is the enterprise API management platform: developer portals, monetization, analytics, advanced policies and hybrid deployment. **Cloud Endpoints** (ESPv2 proxy) suits APIs on GKE or VMs.

KEY POINTS

- API Gateway calls backends using a service account and ID tokens
- API keys identify projects; they are not strong authentication
- Apigee X runs in Google Cloud; Apigee hybrid runs runtimes in your clusters
- Global ALB + Cloud Armor in front of Cloud Run is often enough for simple APIs

OPENAPI.YAML

```yaml
swagger: "2.0"
info: { title: shop-api, version: "1.0" }
x-google-backend:
  address: https://api-abc123-uc.a.run.app
securityDefinitions:
  auth0:
    type: oauth2
    flow: implicit
    authorizationUrl: ""
    x-google-issuer: https://acme.auth0.com/
    x-google-jwks_uri: https://acme.auth0.com/.well-known/jwks.json
    x-google-audiences: shop-api
paths:
  /orders:
    get:
      operationId: listOrders
      security: [{ auth0: [] }]
      responses: { "200": { description: OK } }
```

CODE EXPLAINED

- → `x-google-backend` routes every path to the Cloud Run service
- → The security definition validates JWTs from the issuer
- → Configs are immutable: create v2 and update the gateway
- → The gateway SA needs `run.invoker` on the backend

AWS / AZURE EQUIVALENT

- ↔ **AWS** API Gateway HTTP / REST APIs (aws-22)
- ↔ **Azure** API Management (az-22)

COST CHECK

- $ API Gateway: per million calls after a free tier
- $ Apigee: subscription or pay-as-you-go environments (much higher)
- $ An ALB in front of Cloud Run may cost less for simple routing

CREATE GATEWAY

```
gcloud api-gateway apis create shop-api
gcloud api-gateway api-configs create v1 --api=shop-api \
    --openapi-spec=openapi.yaml \
    --backend-auth-service-account=gw@shop-prod-4821.iam.gserviceaccount.com
gcloud api-gateway gateways create shop-gw --api=shop-api \
    --api-config=v1 --location=us-central1
gcloud api-gateway gateways describe shop-gw --location=us-central1 \
    --format="value(defaultHostname)"
```

SECURITY CHECK

- ⚠ Require JWT / ID tokens, not API keys alone, for user data
- ⚠ Keep backends private (`--no-allow-unauthenticated`) so only the gateway can call them
- ⚠ Restrict API keys by API and by referrer / IP

PICK AN OPTION

NEED	USE
Simple managed gateway for serverless	API Gateway
Developer portal, monetization, analytics	Apigee
APIs on GKE / VMs with a sidecar proxy	Cloud Endpoints (ESPv2)
Just routing + WAF	Global ALB + Cloud Armor

AUTH OPTIONS

METHOD	USE
API key	Identify calling project / app
Google ID token	Service-to-service
JWT (any issuer)	User auth (Auth0, Firebase, Okta)
Apigee OAuth policies	Full OAuth server

DO & DON'T

- ✓ **Do** Version configs and roll gateways forward
- ✓ **Do** Keep the OpenAPI spec in the repo with the code
- ✗ **Don't** Treat API keys as authentication for sensitive data
- ✗ **Don't** Choose Apigee for one small internal API

LOCAL TESTING

- ⌂ Validate the spec with `swagger-cli validate openapi.yaml`
- ⌂ Test JWT validation with tokens from a dev tenant
- ⌂ Run the backend locally and test paths with curl

PRODUCTION CHECKLIST

- ☐ JWT or ID-token auth on every route
- ☐ Private backends; gateway SA as the only invoker
- ☐ Quotas configured; logs and metrics monitored

Sheet 23

Eventarc, Workflows & Scheduler

Eventarc triggers · CloudEvents · event sources (direct, audit logs, Pub/Sub) · Workflows (YAML orchestration) · Cloud Scheduler · Cloud Tasks

EVENTS AND ORCHESTRATION

Eventarc routes events from Google services (directly, via **Cloud Audit Logs**, or Pub/Sub) and third parties to Cloud Run, GKE, Workflows or functions as **CloudEvents**. **Workflows** orchestrates steps (HTTP calls, connectors, conditions, retries, parallel branches) in YAML. **Cloud Scheduler** runs cron jobs, and **Cloud Tasks** queues HTTP calls with rate limiting.

KEY POINTS

- Eventarc Advanced adds a message bus with routing and transformation
- Workflows connectors call Google APIs with built-in auth and retries
- Callbacks let Workflows wait for human approval or external events
- Cloud Tasks gives explicit, rate-limited dispatch to one handler

TRIGGER AND SCHEDULE

```
gcloud eventarc triggers create on-upload --location=us-central1 \
    --destination-run-service=thumbnailer \
    --destination-run-region=us-central1 \
    --event-filters="type=google.cloud.storage.object.v1.finalized" \
    --event-filters="bucket=acme-uploads-prod" \
    --service-account=events@shop-prod-4821.iam.gserviceaccount.com

gcloud workflows deploy order-flow --location=us-central1 \
    --source=order-flow.yaml \
    --service-account=wf@shop-prod-4821.iam.gserviceaccount.com

gcloud scheduler jobs create pubsub nightly-report --location=us-central1 \
    --schedule="0 6 * * 1-5" --time-zone="America/Chicago" \
    --topic=reports --message-body='{"type":"daily"}'
```

CODE EXPLAINED

→ The trigger sends finalized-object events to the thumbnailer

→ Workflows calls Cloud Run with an OIDC token and retries

→ The Pub/Sub connector publishes the ship event

→ Scheduler publishes a weekday report message at 6 AM Chicago time

ORDER-FLOW.YAML

```
main:
  params: [input]
  steps:
    - charge:
        try:
          call: http.post
          args:
            url: https://payments-abc123-uc.a.run.app/charge
            auth: { type: OIDC }
            body: { orderId: ${input.orderId} }
          result: payment
        retry: ${http.default_retry}
    - ship:
        call: googleapis.pubsub.v1.projects.topics.publish
        args:
          topic: projects/shop-prod-4821/topics/ship
          body:
            messages: [{ data: ${base64.encode(json.encode(input))} }]
```

AWS / AZURE EQUIVALENT

↔ **AWS** EventBridge, Step Functions, EventBridge Scheduler (aws-23)

↔ **Azure** Event Grid, Logic Apps / Durable Functions (az-23)

COST CHECK

$ Eventarc: Google-source events free; Pub/Sub delivery bills as Pub/Sub

$ Workflows: per 1,000 steps (internal vs external)

$ Cloud Tasks / Scheduler: free tiers, then per operation / job

PICK A TOOL

NEED	USE
React to Google service events	Eventarc
Multi-step process with retries	Workflows
Cron	Cloud Scheduler
Rate-limited calls to one handler	Cloud Tasks
Fan-out messaging	Pub/Sub
Data pipelines	Dataflow / Cloud Composer

EVENTARC SOURCES

SOURCE	EXAMPLE
Direct	Cloud Storage, Pub/Sub, Firestore
Cloud Audit Logs	Any API method (e.g. SetIamPolicy)
Third party	Partner channels
Custom	Your apps publishing events

DO & DON'T

✓ **Do** Give each trigger and workflow its own service account

✓ **Do** Use connectors instead of raw HTTP to Google APIs

✗ **Don't** Chain functions with direct calls when a workflow fits

✗ **Don't** Forget that events can be delivered more than once

LOCAL TESTING

⌂ Send sample CloudEvents to a local service with curl

⌂ Workflows has no local emulator: test in a dev project

⌂ Pub/Sub emulator for message-driven flows

PRODUCTION CHECKLIST

- ☐ Dead-letter and retry policies on triggers
- ☐ Workflow execution failures alerted
- ☐ Schedules in IaC with explicit time zones

Pub/Sub

GLOBAL MESSAGING

Pub/Sub decouples producers and consumers. Publishers send messages to a **topic**; each **subscription** gets a copy and delivers by **pull** or **push** (HTTPS). Messages must be **acked** before the **ack deadline**, or they are redelivered. **Dead-letter topics** catch poison messages, **ordering keys** keep per-key order, and **BigQuery** / **Cloud Storage** subscriptions write directly to storage.

KEY POINTS

- At-least-once delivery by default; exactly-once is available for pull
- Retention: unacked messages up to 7 days (topic retention up to 31)
- Seek replays messages to a timestamp or snapshot
- Filters deliver only matching messages to a subscription

TOPICS AND SUBSCRIPTIONS

```
gcloud pubsub topics create orders
gcloud pubsub topics create orders-dlq
gcloud pubsub subscriptions create orders-worker --topic=orders \
    --ack-deadline=60 --dead-letter-topic=orders-dlq \
    --max-delivery-attempts=5 --enable-message-ordering \
    --min-retry-delay=10s --max-retry-delay=600s
gcloud pubsub subscriptions create orders-push --topic=orders \
    --push-endpoint=https://audit-abc123-uc.a.run.app/ \
    --push-auth-service-account=push@shop-prod-4821.iam.gserviceaccount.com
gcloud pubsub topics publish orders --message='{"id":"o-1001"}' \
    --ordering-key=customer-42 --attribute=type=created
```

CODE EXPLAINED

- → After 5 failed deliveries the message goes to the dead-letter topic
- → Push subscriptions call Cloud Run with an authenticated ID token
- → Ordering keys keep one customer's messages in order
- → Streaming pull receives messages as they arrive

PUBLISH AND PULL (PYTHON)

```
import json
from google.cloud import pubsub_v1

pub = pubsub_v1.PublisherClient(
    publisher_options=pubsub_v1.types.PublisherOptions(
        enable_message_ordering=True))
topic = pub.topic_path("shop-prod-4821", "orders")
pub.publish(topic, json.dumps({"id": "o-1001"}).encode(),
            ordering_key="customer-42", type="created").result()

sub = pubsub_v1.SubscriberClient()
path = sub.subscription_path("shop-prod-4821", "orders-worker")

def handle(msg):
    process(json.loads(msg.data))
    msg.ack()           # nack() or no ack → redelivery

future = sub.subscribe(path, callback=handle)
future.result()         # blocks; streaming pull
```

AWS / AZURE EQUIVALENT

- ↔ **AWS** SQS + SNS (aws-24)
- ↔ **Azure** Service Bus, Event Hubs (az-24)

COST CHECK

- $ Throughput billed per TiB published and delivered (first 10 GiB / month free)
- $ Retained acked messages and snapshots bill for storage
- $ BigQuery subscriptions avoid paying for a separate writer service

LIMITS & QUOTAS

- ≤ Message size up to 10 MB
- ≤ Ack deadline 10 s to 600 s (extend with modifyAckDeadline)
- ≤ Publish / subscribe throughput quotas per project and region

DELIVERY TYPES

TYPE	USE
Pull (streaming)	Workers that control flow
Push	HTTPS endpoint (Cloud Run)
BigQuery subscription	Write straight to a table
Cloud Storage subscription	Batch files to a bucket

RELIABILITY SETTINGS

SETTING	PURPOSE
Ack deadline	Processing time before redelivery
Retry policy	Backoff between attempts
Dead-letter topic	Poison messages
Exactly-once	No redelivery after successful ack
Ordering key	Per-key ordering

DO & DON'T

- ✓ **Do** Make subscribers idempotent
- ✓ **Do** Add a dead-letter topic to every subscription
- ✗ **Don't** Ack before processing finishes
- ✗ **Don't** Use one subscription for two different consumers (each needs its own)

LOCAL TESTING

- ⌂ `gcloud beta emulators pubsub start` + `PUBSUB_EMULATOR_HOST`
- ⌂ Client libraries switch to the emulator automatically
- ⌂ Seek a dev subscription to replay test messages

PRODUCTION CHECKLIST

- ☐ DLQ with alerting on its backlog
- ☐ Oldest-unacked-message age alerts
- ☐ Push endpoints authenticated with OIDC

Artifact Registry

Docker, npm, Maven, Python, Go… repositories · standard, remote & virtual repos · gcloud auth configure-docker · vulnerability scanning · cleanup policies · Container Registry is shut down

STORING IMAGES AND PACKAGES

Artifact Registry stores container images and language packages in regional or multi-regional **repositories**. **Remote repositories** cache upstreams (Docker Hub, npm, PyPI, Maven Central), and **virtual repositories** combine several behind one URL. **Artifact Analysis** scans images for vulnerabilities, and **cleanup policies** delete old versions. Container Registry (gcr.io) has been replaced by Artifact Registry.

KEY POINTS

- Image paths: `LOCATION-docker.pkg.dev/PROJECT/REPO/IMAGE:TAG`
- gcr.io domains can be served by Artifact Registry repositories
- Use digests (`@sha256:…`) for immutable deploys
- Immutable tags can be enforced per repository

REPOS, PUSH, SCAN, CLEAN UP

```
gcloud artifacts repositories create apps --repository-format=docker \
    --location=us-central1 --immutable-tags \
    --description="Shop container images"
gcloud auth configure-docker us-central1-docker.pkg.dev

IMG=us-central1-docker.pkg.dev/shop-prod-4821/apps/api
docker build -t $IMG:1.4.2 .
docker push $IMG:1.4.2
gcloud builds submit --tag=$IMG:1.4.3 .        # or build in the cloud

gcloud artifacts docker images list $IMG
gcloud artifacts docker images describe $IMG:1.4.2 \
    --show-package-vulnerability

gcloud artifacts repositories set-cleanup-policies apps \
    --location=us-central1 --policy=cleanup.json --no-dry-run

gcloud artifacts repositories create pypi-proxy --location=us-central1 \
    --repository-format=python --mode=remote-repository \
    --remote-python-repo=PYPI
gcloud artifacts repositories add-iam-policy-binding apps \
    --location=us-central1 --role=roles/artifactregistry.reader \
    --member="serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com"
```

CODE EXPLAINED

- → Immutable tags stop a release tag from being overwritten
- → `configure-docker` adds a credential helper for the registry host
- → Scanning results show per-package vulnerabilities
- → The remote repo caches PyPI packages for builds

AWS / AZURE EQUIVALENT

- ↔ **AWS** ECR + pull-through cache (aws-25)
- ↔ **Azure** Azure Container Registry (az-25)

COST CHECK

- $ Storage per GB per month (first 0.5 GB free)
- $ Egress across regions and to the internet bills per GB
- $ Cleanup policies keep old images from piling up

SECURITY CHECK

- ⚠ Grant `artifactregistry.reader` to runtimes and `writer` only to CI
- ⚠ Turn on vulnerability scanning and gate deploys with Binary Authorization
- ⚠ Enable immutable tags for release repositories

LIMITS & QUOTAS

- ≤ Request quotas per project per region (read / write per minute)
- ≤ Docker layer size limits apply (very large layers fail)
- ≤ Cleanup policies run asynchronously (not instant)

REPOSITORY MODES

MODE	USE
Standard	Your images / packages
Remote	Cache Docker Hub, PyPI, npm, Maven
Virtual	One URL over several repos

FORMATS

FORMAT	CLIENT
Docker / OCI	docker, helm
npm	npm, pnpm, yarn
Python	pip, twine
Maven / Gradle	mvn, gradle
Go, Apt, Yum, generic	Various

DO & DON'T

- ✓ **Do** Deploy by digest, not mutable tags
- ✓ **Do** Put a remote repo in front of Docker Hub to avoid rate limits
- ✗ **Don't** Keep using gcr.io paths without migrating
- ✗ **Don't** Grant `writer` to runtime service accounts

LOCAL TESTING

- ⌂ `docker build` + `docker run` locally before pushing
- ⌂ Trivy / Grype scan images locally
- ⌂ Point pip / npm at the remote repo to test caching

PRODUCTION CHECKLIST

- ☐ Immutable tags + scanning on release repos
- ☐ Cleanup policies tested in dry-run
- ☐ Binary Authorization requiring scanned, attested images

GKE Basics

Autopilot vs Standard · regional clusters · private clusters & authorized networks · Workload Identity Federation for GKE · release channels · `gcloud container clusters get-credentials`

MANAGED KUBERNETES

Google Kubernetes Engine (GKE) runs Kubernetes with a Google-managed control plane. **Autopilot** manages nodes too and bills per Pod request; **Standard** lets you manage node pools. Use **regional** clusters for HA, **private nodes** with limited control plane access, **Workload Identity Federation for GKE** for Pod permissions, and a **release channel** for automatic upgrades.

KEY POINTS

- Autopilot enforces security best practices and Pod resource requests
- DNS-based control plane endpoint can replace IP allowlists
- Gateway API replaces many Ingress use cases
- Use kubectl with `gke-gcloud-auth-plugin`

SECURITY CHECK

- ⚠ Private nodes and a restricted control plane endpoint
- ⚠ Workload Identity Federation for GKE, never node SA keys
- ⚠ Enable Binary Authorization and GKE security posture dashboard

LIMITS & QUOTAS

- ≤ Standard clusters default to a 15,000-node limit; Autopilot has its own Pod limits
- ≤ Pods per node default 110 (Standard), set by Pod CIDR size
- ≤ Cluster management fee per cluster-hour (free tier credit covers one)

CREATE, CONNECT, GRANT PERMISSIONS TO PODS

```
gcloud container clusters create-auto shop --region=us-central1 \
    --network=vpc-shop --subnetwork=gke-usc1 \
    --cluster-secondary-range-name=pods \
    --services-secondary-range-name=services \
    --enable-private-nodes --release-channel=regular
gcloud container clusters get-credentials shop --region=us-central1
kubectl get nodes

# Pod identity: grant a role directly to a Kubernetes service account
kubectl create namespace api
kubectl create serviceaccount api -n api
POOL=projects/123456789012/locations/global/workloadIdentityPools
WIP=$POOL/shop-prod-4821.svc.id.goog
PRINCIPAL=principal://iam.googleapis.com/$WIP/subject/ns/api/sa/api
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --role=roles/secretmanager.secretAccessor --member="$PRINCIPAL"

kubectl -n api create deployment api \
    --image=us-central1-docker.pkg.dev/shop-prod-4821/apps/api:1.4.2
kubectl -n api set serviceaccount deployment/api api
kubectl -n api set resources deployment/api --requests=cpu=250m,memory=512Mi
```

CODE EXPLAINED

- → The Autopilot cluster uses the subnet's secondary ranges
- → Private nodes have no external IPs
- → The `principal://` member grants a role to one Kubernetes SA
- → Autopilot bills on the Pod resource requests

AWS / AZURE EQUIVALENT

- ↔ **AWS** EKS + Pod Identity (aws-26)
- ↔ **Azure** AKS + workload identity (az-26)

COST CHECK

- $ Autopilot: per vCPU, memory and storage requested by Pods
- $ Standard: per node VM; spread Spot node pools for batch
- $ One zonal or Autopilot cluster's management fee is covered by a monthly free tier credit

AUTOPILOT VS STANDARD

ASPECT	AUTOPILOT	STANDARD
Nodes	Google-managed	You manage node pools
Billing	Pod requests	Node VMs
Security defaults	Enforced	You configure
DaemonSets / privileged	Restricted	Allowed
Best for	Most workloads	Special hardware / tuning

RELEASE CHANNELS

CHANNEL	UPGRADES
Rapid	Newest versions first
Regular	Default balance
Stable	Most tested
Extended	Longer support (fee)

DO & DON'T

- ✓ **Do** Start with Autopilot
- ✓ **Do** Set resource requests on every container
- ✗ **Don't** Expose the control plane to 0.0.0.0/0
- ✗ **Don't** Give Pods the node service account's permissions

LOCAL TESTING

- ⌂ kind or minikube for manifests
- ⌂ Skaffold / Cloud Code for dev loops against GKE
- ⌂ `kubectl --dry-run=server` validates against the cluster

PRODUCTION CHECKLIST

- ☐ Regional, private, release channel set
- ☐ Workload Identity Federation per workload
- ☐ Binary Authorization and Pod Security enforced

GKE Operations

Gateway API & Ingress · autoscaling (HPA, VPA, cluster autoscaler, NAP) · maintenance windows · Config Sync & Policy Controller · Backup for GKE · GKE Enterprise / fleets

RUNNING GKE DAY TO DAY

Expose services with the **Gateway API** (GKE controller provisions Google load balancers) or Ingress. Scale Pods with **HPA** / **VPA** and nodes with the **cluster autoscaler** or node auto-provisioning (built in on Autopilot). Control upgrades with **maintenance windows and exclusions**. **Config Sync** applies Git-stored config and **Policy Controller** enforces policies across a **fleet**. **Backup for GKE** protects workloads and volumes.

KEY POINTS

- Gateway classes: `gke-l7-global-external-managed`, `gke-l7-rilb` …
- PodDisruptionBudgets keep upgrades from taking everything down
- Managed Service for Prometheus collects metrics without running Prometheus
- Surge upgrades and blue-green node upgrades are configurable

GATEWAY, HTTPROUTE, HPA

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata: { name: shop, namespace: api }
spec:
  gatewayClassName: gke-l7-global-external-managed
  listeners:
    - { name: http, protocol: HTTP, port: 80 }
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: api, namespace: api }
spec:
  parentRefs: [{ name: shop }]
  rules:
    - matches: [{ path: { type: PathPrefix, value: /api } }]
      backendRefs: [{ name: api, port: 80 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: api, namespace: api }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: api }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 60 }
```

CODE EXPLAINED

- → The Gateway provisions a global external Application Load Balancer
- → The HTTPRoute sends `/api` traffic to the api Service
- → HPA keeps average CPU near 60% across 3–20 Pods
- → The backup plan protects every namespace nightly for 14 days

AWS / AZURE EQUIVALENT

- ↔ **AWS** AWS Load Balancer Controller, Karpenter, EKS add-ons (aws-27)
- ↔ **Azure** AKS app routing, KEDA, AKS backup (az-27)

COST CHECK

- $ Each Gateway / Ingress creates billable load balancer pieces
- $ Backup for GKE: per protected Pod + backup storage
- $ Right-size requests (VPA recommendations) to cut Autopilot bills

BACKUP FOR GKE

```
gcloud beta container backup-restore backup-plans create shop-daily \
    --location=us-central1 \
    --cluster=projects/shop-prod-4821/locations/us-central1/clusters/shop \
    --all-namespaces --include-volume-data --cron-schedule="0 3 * * *" \
    --backup-retain-days=14
```

LIMITS & QUOTAS

- ≤ Maintenance exclusions are limited in length and number
- ≤ HPA needs resource requests to compute utilization
- ≤ Gateway / Ingress map to load balancer quotas (forwarding rules, backend services)

SCALING LAYERS

LAYER	SCALES
HPA	Pod replicas on metrics
VPA	Pod requests / limits
Cluster autoscaler	Nodes in a pool
Node auto-provisioning	Creates node pools
Autopilot	Nodes handled for you

FLEET TOOLS

TOOL	DOES
Config Sync	GitOps for cluster config
Policy Controller	OPA Gatekeeper policies
Cloud Service Mesh	mTLS, traffic management
Fleet observability	Multi-cluster views

DO & DON'T

- ✓ **Do** Use PodDisruptionBudgets for critical Deployments
- ✓ **Do** Set maintenance windows that match low traffic
- ✗ **Don't** Run without requests (HPA and Autopilot both rely on them)
- ✗ **Don't** Upgrade on the Rapid channel in production

LOCAL TESTING

- ⌂ `kubectl apply --dry-run=server -f gateway.yaml`
- ⌂ kind + Gateway API CRDs for local routing tests
- ⌂ Restore a Backup for GKE plan to a test cluster

PRODUCTION CHECKLIST

- ☐ Gateway API with Cloud Armor via GCPBackendPolicy
- ☐ Maintenance windows + PDBs
- ☐ Backup for GKE tested by restore drills

Cloud Storage Basics

buckets & objects · `gcloud storage cp` / `rsync` / `ls` / `rm` · location types (region, dual-region, multi-region) · storage classes · Autoclass · client libraries

OBJECT STORAGE

Cloud Storage keeps **objects** in **buckets** with globally unique names. A bucket's **location type** (region, dual-region, multi-region) sets availability and latency. **Storage classes** (Standard, Nearline, Coldline, Archive) trade storage price for retrieval cost and minimum storage duration; **Autoclass** moves objects between classes automatically. All classes have millisecond access.

KEY POINTS

- Object names can contain `/`, but the namespace is flat (folders are simulated, unless hierarchical namespace is on)
- `gcloud storage` does parallel composite uploads automatically
- Strong read-after-write consistency for objects and listings
- Hierarchical namespace buckets add real folders for analytics workloads

SECURITY CHECK

- ⚠ Keep public access prevention enforced (org policy)
- ⚠ Use uniform bucket-level access, not object ACLs
- ⚠ Grant bucket-level IAM to service accounts, not project-wide storage roles

LIMITS & QUOTAS

- ≤ Object size up to 5 TiB
- ≤ About 1 write per second to the same object name
- ≤ Bucket names are global and DNS-compatible (3–63 characters)

BUCKETS AND OBJECTS

```
gcloud storage buckets create gs://acme-media-prod --location=us-central1 \
    --default-storage-class=STANDARD --uniform-bucket-level-access \
    --public-access-prevention
gcloud storage buckets create gs://acme-logs-archive --location=US \
    --enable-autoclass

gcloud storage cp report.pdf gs://acme-media-prod/2026/
gcloud storage cp -r ./site gs://acme-site/
gcloud storage rsync ./build gs://acme-site --recursive \
    --delete-unmatched-destination-objects
gcloud storage ls -l gs://acme-media-prod/2026/
gcloud storage objects update gs://acme-media-prod/2026/report.pdf \
    --storage-class=NEARLINE
gcloud storage rm gs://acme-media-prod/tmp/**
gcloud storage du -s gs://acme-media-prod
```

CODE EXPLAINED

- → UBLA + public access prevention lock the bucket down
- → Autoclass picks classes per object based on access
- → `rsync` with delete mirrors a build folder
- → The double-star wildcard matches every object under a prefix

AWS / AZURE EQUIVALENT

- ↔ **AWS** S3 buckets, storage classes, Intelligent-Tiering (aws-28)
- ↔ **Azure** Blob Storage accounts, access tiers (az-28)

PYTHON CLIENT

```
from google.cloud import storage
bucket = storage.Client().bucket("acme-media-prod")
bucket.blob("2026/report.pdf").upload_from_filename("report.pdf")
for blob in bucket.list_blobs(prefix="2026/"):
    print(blob.name, blob.size, blob.storage_class)
data = bucket.blob("2026/notes.txt").download_as_text()
```

COST CHECK

- $ Storage per GB-month by class + operations + retrieval (colder classes)
- $ Minimum durations: Nearline 30, Coldline 90, Archive 365 days
- $ Egress and cross-region replication bill per GB

STORAGE CLASSES

CLASS	MIN DURATION	USE
Standard	None	Hot data
Nearline	30 days	Monthly access
Coldline	90 days	Quarterly access
Archive	365 days	Yearly / compliance

LOCATION TYPES

TYPE	EXAMPLE
Region	us-central1
Dual-region	NAM4, or custom pair
Multi-region	US, EU, ASIA

DO & DON'T

- ✓ **Do** Use Autoclass when access patterns are unknown
- ✓ **Do** Prefix object names to spread load and organize lifecycle
- ✗ **Don't** Use Archive for data you read often (retrieval fees)
- ✗ **Don't** Mount buckets as a POSIX file system for databases

LOCAL TESTING

- ⌂ `gcloud storage` against a dev bucket
- ⌂ fake-gcs-server in Docker emulates the JSON API
- ⌂ Unit tests mock the storage client

PRODUCTION CHECKLIST

- ☐ UBLA, public access prevention, bucket IAM
- ☐ Lifecycle / Autoclass for cost
- ☐ Soft delete and versioning set for critical buckets

Cloud Storage Security & Features

uniform bucket-level access · public access prevention · signed URLs · versioning & soft delete · lifecycle rules · retention policies & Bucket Lock

PROTECTING AND MANAGING OBJECTS

Uniform bucket-level access makes IAM the only access control. **Public access prevention** blocks `allUsers` grants. **Signed URLs** give time-limited access without IAM. **Object versioning** and **soft delete** (7 days by default) recover deletes and overwrites; **lifecycle rules** change classes or delete objects. **Retention policies** with **Bucket Lock** give WORM compliance.

KEY POINTS

- Signed URLs (V4) are valid for up to 7 days
- Signing with an attached service account uses the IAM signBlob API (no key file)
- Static websites: serve through a load balancer backend bucket (sheet 14)
- CMEK and VPC Service Controls protect sensitive buckets

LIFECYCLE, VERSIONING, RETENTION

```
gcloud storage buckets update gs://acme-media-prod --versioning \
    --soft-delete-duration=14d
gcloud storage buckets update gs://acme-media-prod \
    --lifecycle-file=lifecycle.json
gcloud storage buckets update gs://acme-audit --retention-period=2557d
gcloud storage buckets update gs://acme-audit --lock-retention-period

gcloud storage ls --soft-deleted gs://acme-media-prod/2026/
gcloud storage restore gs://acme-media-prod/2026/report.pdf#1767225600000000

gcloud storage sign-url gs://acme-media-prod/2026/report.pdf --duration=15m \
    --impersonate-service-account=signer@shop-prod-4821.iam.gserviceaccount.com
```

CODE EXPLAINED

- → Soft delete keeps deleted objects restorable for 14 days
- → Logs move to Nearline after 30 days, delete after a year
- → Locking a retention policy is permanent
- → The signed URL works for 15 minutes for anyone holding it

LIFECYCLE.JSON

```
{"rule": [
  {"action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
   "condition": {"age": 30, "matchesPrefix": ["logs/"]}},
  {"action": {"type": "Delete"},
   "condition": {"age": 365, "matchesPrefix": ["logs/"]}},
  {"action": {"type": "Delete"},
   "condition": {"isLive": false, "daysSinceNoncurrentTime": 30}}
]}
```

AWS / AZURE EQUIVALENT

↔ **AWS**	Block Public Access, presigned URLs, Object Lock (aws-29)
↔ **Azure**	SAS tokens, soft delete, immutability (az-29)

COST CHECK

- $ Versioning and soft delete keep billing for old / deleted data
- $ Lifecycle actions are free (class-change operations bill)
- $ Early deletion of colder classes still bills the minimum duration

SIGNED URL IN PYTHON

```
import datetime
from google.cloud import storage
blob = storage.Client().bucket("acme-media-prod").blob("2026/report.pdf")
url = blob.generate_signed_url(version="v4", method="GET",
    expiration=datetime.timedelta(minutes=15))
```

SECURITY CHECK

- ⚠ Enforce public access prevention org-wide (`storage.publicAccessPrevention`)
- ⚠ Keep soft delete on for buckets holding irreplaceable data
- ⚠ Prefer short signed URL expiry (minutes, not days)

ACCESS CONTROLS

CONTROL	USE
Uniform bucket-level access	IAM only (recommended)
Public access prevention	Block public grants
Signed URL / policy document	Temporary access, uploads
IAM Conditions	Prefix-based grants
VPC Service Controls	Perimeter around buckets

DATA PROTECTION

FEATURE	PROTECTS AGAINST
Soft delete	Accidental delete (default 7 days)
Object versioning	Overwrites
Retention policy	Early deletion
Bucket Lock	Changes to retention (WORM)
Object holds	Legal / event holds

DO & DON'T

- ✓ **Do** Generate signed URLs server-side with short expiry
- ✓ **Do** Pair versioning with a noncurrent-version lifecycle rule
- ✗ **Don't** Grant `allUsers` to serve a website (use a load balancer)
- ✗ **Don't** Lock a retention policy before you are sure of the period

LOCAL TESTING

- ⌂ Test lifecycle rules on a scratch bucket with small ages
- ⌂ Sign URLs locally with impersonated ADC
- ⌂ fake-gcs-server for upload / download flows

PRODUCTION CHECKLIST

- ☐ Public access prevention enforced by org policy
- ☐ Soft delete + versioning on critical data
- ☐ Retention / Bucket Lock for compliance data

Disks, Filestore & Backup

Persistent Disk vs Hyperdisk · regional disks · snapshot schedules · Filestore (NFS) · Cloud Storage FUSE · Backup and DR Service

BLOCK, FILE AND BACKUPS

VM block storage comes as **Persistent Disk** (balanced, SSD, standard) or **Hyperdisk** (Balanced, Throughput, Extreme, ML), where you tune IOPS and throughput separately. **Regional disks** replicate across two zones. **Filestore** provides managed NFS shares. **Cloud Storage FUSE** mounts buckets for read-heavy workloads. **Backup and DR Service** manages backup plans, vaults and app-consistent backups.

KEY POINTS

- Disks can grow online, never shrink
- Hyperdisk Balanced High Availability replicates synchronously across zones
- Local SSDs are fast but lose data when the VM stops
- Backup vaults can be immutable and indelible for ransomware protection

DISKS, SNAPSHOTS, FILESTORE, BACKUPS

```
gcloud compute disks create data-1 --zone=us-central1-a \
    --type=hyperdisk-balanced --size=500GB \
    --provisioned-iops=6000 --provisioned-throughput=290
gcloud compute instances attach-disk web-1 --disk=data-1 \
    --zone=us-central1-a
gcloud compute disks resize data-1 --zone=us-central1-a --size=1TB

gcloud compute disks create db-ha --region=us-central1 --type=pd-ssd \
    --size=200GB --replica-zones=us-central1-a,us-central1-b

gcloud filestore instances create shared --zone=us-central1-a \
    --tier=BASIC_HDD --file-share=name=vol1,capacity=1TB \
    --network=name=vpc-shop
# On a VM: sudo mount 10.10.0.20:/vol1 /mnt/shared

gcloud backup-dr backup-vaults create vault-prod --location=us-central1 \
    --backup-min-enforced-retention=30d
VAULT=projects/shop-prod-4821/locations/us-central1/backupVaults/vault-prod
RULE=rule-id=daily,recurrence=DAILY,retention-days=30
gcloud backup-dr backup-plans create daily-vm --location=us-central1 \
    --backup-vault=$VAULT --resource-type=compute.googleapis.com/Instance \
    --backup-rule=$RULE,backup-window-start=2,backup-window-end=6
```

CODE EXPLAINED

- → Hyperdisk sets IOPS and throughput independently of size
- → The regional disk survives the loss of one zone
- → Filestore exposes an NFS share on the VPC
- → The vault enforces at least 30 days of retention

AWS / AZURE EQUIVALENT

↔ **AWS**	EBS, EFS, AWS Backup (aws-30)
↔ **Azure**	Managed Disks, Azure Files, Azure Backup (az-30)

COST CHECK

- $ Disks bill per GB provisioned (Hyperdisk adds IOPS / throughput)
- $ Filestore bills per provisioned TiB even when empty
- $ Snapshots and backup vaults bill per GB stored

PICK STORAGE

NEED	USE
VM boot / data disk	Persistent Disk / Hyperdisk
Zone-failure tolerance	Regional PD / Hyperdisk HA
Shared POSIX files	Filestore
Objects, read-heavy mounts	Cloud Storage (+ FUSE)
Scratch	Local SSD

BACKUP OPTIONS

TOOL	SCOPE
Snapshot schedules	Disks
Machine images	Whole VMs
Backup and DR Service	VMs, disks, Cloud SQL, apps
Backup for GKE	Clusters (sheet 27)
Cloud SQL backups / PITR	Databases (sheet 31)

DO & DON'T

- ✓ **Do** Use Hyperdisk when you need tuned IOPS
- ✓ **Do** Store backups in a locked vault
- ✗ **Don't** Keep important data only on Local SSD
- ✗ **Don't** Overprovision Filestore capacity you never use

LOCAL TESTING

- ⌂ `fio` benchmarks disk IOPS on a test VM
- ⌂ Restore a snapshot to a new disk to test recovery
- ⌂ Mount a test bucket with gcsfuse to check access patterns

PRODUCTION CHECKLIST

- ☐ Backup plans with locked vaults
- ☐ Regional disks for stateful single VMs
- ☐ Restore drills scheduled and documented

Cloud SQL & AlloyDB

MANAGED RELATIONAL DATABASES

Cloud SQL runs managed MySQL, PostgreSQL and SQL Server with backups, patching, **regional HA** (standby in another zone) and **read replicas**. **Enterprise Plus** adds faster failover and data cache. Connect with **private IP** and the **Cloud SQL Auth Proxy** or language **connectors** (IAM-authorized, TLS). **AlloyDB** is a PostgreSQL-compatible engine for higher performance and analytics.

KEY POINTS

- IAM database authentication lets service accounts log in without passwords
- PITR uses transaction logs (binary logs / WAL)
- Maintenance windows and deny periods control patching
- Cross-region replicas support DR and promotion

SECURITY CHECK

- ⚠ Private IP only; no authorized networks of 0.0.0.0/0
- ⚠ Use IAM database auth or Secret Manager passwords, never hard-coded
- ⚠ Require SSL / connectors (`--ssl-mode=ENCRYPTED_ONLY`)

LIMITS & QUOTAS

- ≤ Storage up to 64 TB per instance
- ≤ Max connections depend on the tier and engine flags
- ≤ Custom machine shapes need the Enterprise edition

CREATE, CONNECT, RECOVER

```
gcloud sql instances create shop-pg --database-version=POSTGRES_16 \
    --edition=ENTERPRISE --tier=db-custom-2-8192 --region=us-central1 \
    --availability-type=REGIONAL --storage-auto-increase \
    --network=projects/shop-prod-4821/global/networks/vpc-shop \
    --no-assign-ip --enable-point-in-time-recovery \
    --backup-start-time=03:00 --ssl-mode=ENCRYPTED_ONLY \
    --database-flags=cloudsql.iam_authentication=on
gcloud sql databases create shop --instance=shop-pg
gcloud sql users create api@shop-prod-4821.iam --instance=shop-pg \
    --type=CLOUD_IAM_SERVICE_ACCOUNT
gcloud sql instances create shop-pg-r1 --master-instance-name=shop-pg \
    --region=us-east1

./cloud-sql-proxy --auto-iam-authn --private-ip \
    shop-prod-4821:us-central1:shop-pg &
psql "host=127.0.0.1 user=api@shop-prod-4821.iam dbname=shop"

gcloud sql instances clone shop-pg shop-pg-restore \
    --point-in-time=2026-11-12T09:30:00Z
```

CODE EXPLAINED

- → Regional availability keeps a standby in a second zone
- → The IAM user logs in with its service account token
- → The proxy handles TLS and IAM auth on localhost
- → Cloning to a point in time restores into a new instance

AWS / AZURE EQUIVALENT

- ↔ **AWS** RDS, Aurora, RDS Proxy (aws-31)
- ↔ **Azure** Azure SQL Database, PostgreSQL Flexible Server (az-31)

COST CHECK

- $ Billed per vCPU, memory, storage and backups; HA roughly doubles compute
- $ CUDs discount steady Cloud SQL usage
- $ Read replicas and cross-region replicas each bill as instances

EDITIONS

FEATURE	ENTERPRISE	ENTERPRISE PLUS
Machine shapes	Custom / shared core	Performance-optimized
Failover	Under 60 s	Under 10 s (near-zero downtime maintenance)
Data cache	No	Yes
Log retention for PITR	Up to 7 days	Up to 35 days

CONNECTING

METHOD	USE
Auth Proxy	Local tools, VMs, GKE sidecar
Language connectors	Python, Java, Go, Node apps
Cloud Run integration	`--add-cloudsql-instances`
Private IP direct	Same VPC with SSL

DO & DON'T

- ✓ **Do** Turn on PITR and test restores
- ✓ **Do** Use connectors so credentials never cross the wire
- ✗ **Don't** Open public IP with broad authorized networks
- ✗ **Don't** Size max connections without a pooler for serverless clients

LOCAL TESTING

- ⌂ PostgreSQL / MySQL in Docker for development
- ⌂ Auth Proxy to reach a dev instance from your laptop
- ⌂ Run migrations in CI against a container first

PRODUCTION CHECKLIST

- ☐ Private IP, regional HA, PITR
- ☐ IAM DB auth; deletion protection on
- ☐ Maintenance window set; replica for DR

Firestore, Spanner & Bigtable

Firestore documents & collections · queries & composite indexes · security rules · Spanner (global SQL) · Bigtable (wide-column) · choosing a NoSQL store

NON-RELATIONAL AND GLOBAL DATABASES

Firestore is a serverless document database: **documents** in **collections**, real-time listeners, offline mobile sync and **security rules** for client access. **Spanner** is a horizontally scaling, strongly consistent relational database with SQL, available regionally or globally. **Bigtable** is a wide-column store for huge, low-latency key-range workloads like time series.

KEY POINTS

- Firestore queries need indexes; composite indexes are defined in `firestore.indexes.json`
- Firestore with MongoDB compatibility accepts MongoDB drivers
- Spanner capacity is set in processing units (100 PU minimum)
- Bigtable row-key design decides performance

FIRESTORE (PYTHON)

```
from google.cloud import firestore
db = firestore.Client(database="(default)")
db.collection("orders").document("o-1001").set(
    {"customer": "c-42", "status": "open", "total": 59.9,
     "created": firestore.SERVER_TIMESTAMP})

q = (db.collection("orders")
       .where(filter=firestore.FieldFilter("customer", "==", "c-42"))
       .where(filter=firestore.FieldFilter("status", "==", "open"))
       .order_by("created", direction=firestore.Query.DESCENDING)
       .limit(20))
for doc in q.stream():
    print(doc.id, doc.to_dict())

@firestore.transactional
def close(tx, ref):
    tx.update(ref, {"status": "closed"})
close(db.transaction(), db.collection("orders").document("o-1001"))
```

CODE EXPLAINED

- → `SERVER_TIMESTAMP` is set by the server at write time
- → Two equality filters plus `order_by` need a composite index
- → The transaction retries automatically on contention
- → Spanner starts at 100 processing units (a tenth of a node)

AWS / AZURE EQUIVALENT

- ↔ **AWS** DynamoDB (aws-32)
- ↔ **Azure** Cosmos DB (az-32)

CREATE DATABASES

```
gcloud firestore databases create --location=nam5 \
    --delete-protection --enable-pitr
gcloud spanner instances create shop-span --config=regional-us-central1 \
    --processing-units=100 --description="shop"
gcloud spanner databases create shop --instance=shop-span \
    --ddl="CREATE TABLE Orders (Id STRING(36), Total NUMERIC) PRIMARY KEY (Id)"
gcloud bigtable instances create metrics --display-name=metrics \
    --cluster-config=id=metrics-c1,zone=us-central1-b,nodes=1
```

COST CHECK

- $ Firestore: per document read / write / delete + storage (free daily quota)
- $ Spanner: per processing-unit hour + storage (no scale to zero)
- $ Bigtable: per node-hour + storage

SECURITY CHECK

- ⚠ Never ship Firestore rules that allow `read, write: if true`
- ⚠ Server code uses IAM (service accounts); clients use security rules + Firebase Auth
- ⚠ Turn on point-in-time recovery and delete protection

LIMITS & QUOTAS

- ≤ Firestore document up to 1 MiB
- ≤ About 1 sustained write per second per Firestore document
- ≤ Spanner storage scales with compute: about 1 TB per 100 processing units

PICK A STORE

NEED	USE
Mobile / web app data, real time	Firestore
MongoDB drivers, serverless	Firestore (MongoDB compatibility)
Global relational, strong consistency	Spanner
Huge time series / IoT	Bigtable
Analytics	BigQuery (sheet 33)
Classic relational	Cloud SQL / AlloyDB (sheet 31)

FIRESTORE MODES

ITEM	NOTE
Native mode	Real time, mobile SDKs
Datastore mode	Legacy Datastore apps
Named databases	Several per project
Security rules	Client access control
TTL policies	Auto-delete old docs

DO & DON'T

- ✓ **Do** Design Firestore documents around your queries
- ✓ **Do** Spread Spanner primary keys (avoid monotonically increasing keys)
- ✗ **Don't** Write one Firestore document from many clients per second (counter hot spot)
- ✗ **Don't** Choose Spanner for a small app that fits Cloud SQL

LOCAL TESTING

- ⌂ `gcloud emulators firestore start` + `FIRESTORE_EMULATOR_HOST`
- ⌂ Spanner and Bigtable emulators via `gcloud emulators`
- ⌂ Firebase emulator suite tests security rules

PRODUCTION CHECKLIST

- ☐ PITR and delete protection on
- ☐ Security rules tested in CI
- ☐ Indexes and schema managed as code

BigQuery & Memorystore

ANALYTICS AND CACHING

BigQuery is a serverless data warehouse: load or stream data into **tables** in **datasets** and query with GoogleSQL. **Partition** tables by date and **cluster** by common filters to scan less. Pay **on-demand** per TiB scanned or buy **editions** capacity (slots). **Memorystore** provides managed in-memory caches: **Memorystore for Valkey**, Redis Cluster and Redis, plus Memcached.

KEY POINTS

- Use `--dry_run` to see bytes scanned before running a query
- External tables / BigLake query files in Cloud Storage
- Streaming via the Storage Write API
- Memorystore uses private IP (PSC for Valkey / Redis Cluster)

BIGQUERY

```
bq mk --dataset --location=US shop-prod-4821:sales
bq mk --table --time_partitioning_field=order_date \
    --clustering_fields=customer_id sales.orders schema.json
bq load --source_format=NEWLINE_DELIMITED_JSON sales.orders \
    gs://acme-exports/orders/*.json

bq query --use_legacy_sql=false --dry_run \
  'SELECT customer_id, SUM(total) AS spend
   FROM sales.orders
   WHERE order_date >= "2026-11-01"
   GROUP BY customer_id ORDER BY spend DESC LIMIT 10'
```

CODE EXPLAINED

- → Partitioning by date prunes old data from scans
- → `--dry_run` reports bytes processed without charging
- → Valkey speaks the Redis protocol, so `redis` clients work
- → Cache entries expire after 5 minutes

MEMORYSTORE FOR VALKEY AND A CACHE-ASIDE READ

```
# gcloud memorystore instances create cache --location=us-central1 \
#   --engine-version=VALKEY_8_0 --shard-count=1 \
#   --node-type=shared-core-nano \
#   --psc-auto-connections=network=$VPC_PATH,projectId=shop-prod-4821
import json, redis
r = redis.Redis(host="10.10.0.30", port=6379)

def get_product(pid):
    if (hit := r.get(f"product:{pid}")):
        return json.loads(hit)
    product = load_from_db(pid)
    r.setex(f"product:{pid}", 300, json.dumps(product))
    return product
```

AWS / AZURE EQUIVALENT

- ↔ **AWS** Redshift / Athena, ElastiCache (aws-33)
- ↔ **Azure** Synapse / Fabric, Azure Managed Redis (az-33)

COST CHECK

- $ BigQuery on-demand: per TiB scanned (first 1 TiB per month free) + storage
- $ Editions: slot-hours with autoscaling; commitments discount
- $ Memorystore: per GB / node-hour, billed while running

BIGQUERY COST CONTROLS

CONTROL	EFFECT
`Partition + cluster`	Scan less
`SELECT only needed columns`	Columnar: fewer bytes
`--maximum_bytes_billed`	Fail expensive queries
`Custom quotas`	Cap per user / project per day
`Long-term storage`	Auto discount after 90 days unchanged

MEMORYSTORE OPTIONS

ENGINE	USE
`Valkey`	Default choice, open source
`Redis Cluster`	Sharded Redis
`Redis (instance)`	Simple Redis, legacy apps
`Memcached`	Simple key-value cache

DO & DON'T

- ✓ **Do** Dry-run new queries and set maximum bytes billed
- ✓ **Do** Put TTLs on every cache entry
- ✗ **Don't** `SELECT *` on large tables
- ✗ **Don't** Use Memorystore as the only copy of important data

LOCAL TESTING

- ⌂ BigQuery has no local emulator: use a dev dataset with small samples
- ⌂ Valkey / Redis in Docker for cache code
- ⌂ Unit test SQL with `bq query --dry_run` in CI

PRODUCTION CHECKLIST

- ☐ Partitioned, clustered tables with expiration where suitable
- ☐ Byte limits and quotas per user
- ☐ Cache sized and monitored for evictions

Sheet 34

Terraform on Google Cloud

google & google-beta providers · google_project_service · Cloud Storage state backend · WIF authentication · terraform plan / apply · Cloud Foundation Toolkit modules

THE COMMON IAC CHOICE

Terraform is the most widely used IaC tool for Google Cloud. The `google` provider covers GA features and `google-beta` adds preview ones. Store state in a **Cloud Storage backend** (with built-in locking and versioning), authenticate with ADC or Workload Identity Federation, and reuse **Cloud Foundation Toolkit** / Terraform Google modules. Enable APIs with `google_project_service`.

KEY POINTS

- Provider defaults (`project` , `region`) reduce repetition
- `terraform import` / import blocks adopt existing resources
- `google_*_iam_member` adds bindings; `_iam_policy` replaces the whole policy
- Infrastructure Manager runs Terraform for you (sheet 35)

MAIN.TF

```
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 7.0" }
  }
  backend "gcs" { bucket = "acme-tfstate", prefix = "shop/prod" }
}
provider "google" {
  project = var.project
  region  = "us-central1"
}

resource "google_project_service" "run" {
  service = "run.googleapis.com"
}

resource "google_service_account" "api" {
  account_id = "api"
}

resource "google_cloud_run_v2_service" "api" {
  name       = "api"
  location   = "us-central1"
  ingress    = "INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER"
  depends_on = [google_project_service.run]
  template {
    service_account = google_service_account.api.email
    containers { image = var.image }
  }
}
```

CODE EXPLAINED

- → The GCS backend stores state with locking
- → The API is enabled before the Cloud Run service is created
- → The service runs as its own service account
- → `plan -out` saves the exact plan that `apply` runs

AWS / AZURE EQUIVALENT

- ↔ **AWS** CloudFormation, Terraform AWS provider (aws-34, aws-36)
- ↔ **Azure** Bicep, Terraform azurerm (az-34, az-36)

COST CHECK

- $ Terraform CLI is free; HCP Terraform has paid tiers
- $ State bucket costs are tiny
- $ Infracost estimates plan cost in pull requests

SECURITY CHECK

- ⚠ Turn on versioning for the state bucket and restrict access (state holds secrets)
- ⚠ Prefer `_iam_member` over authoritative `_iam_policy` resources
- ⚠ Run applies from CI with WIF, not personal credentials

WORKFLOW

```
terraform init && terraform fmt -check && terraform validate
terraform plan -out=tfplan -var="project=shop-prod-4821"
terraform apply tfplan
```

LIMITS & QUOTAS

- ≤ API quotas still apply during large applies (use `-parallelism`)
- ≤ Org policy and project quotas can fail applies mid-way
- ≤ One state per environment keeps plans small and fast

KEY RESOURCES

RESOURCE	CREATES
google_project_service	Enables an API
google_service_account	Service account
google_*_iam_member	One IAM binding
google_cloud_run_v2_service	Cloud Run service
google_storage_bucket	Bucket
google_sql_database_instance	Cloud SQL

IAM RESOURCE TYPES

SUFFIX	BEHAVIOR
_iam_member	Adds one member (safe)
_iam_binding	Owns all members of one role
_iam_policy	Owns the entire policy (dangerous)

DO & DON'T

- ✓ **Do** Pin provider major versions
- ✓ **Do** Separate state per environment and per layer
- ✗ **Don't** Mix `_iam_policy` with other IAM resources on the same resource
- ✗ **Don't** Edit Terraform-managed resources in the console

LOCAL TESTING

- ⌂ `terraform plan` with impersonated ADC
- ⌂ tflint + Checkov in pre-commit
- ⌂ `terraform test` for module tests

PRODUCTION CHECKLIST

- ☐ CI applies with WIF and approval gates
- ☐ Versioned, locked-down state bucket
- ☐ Drift checked by scheduled plans

Infrastructure Manager

managed Terraform runs · `gcloud infra-manager deployments apply` · revisions & previews · service account per deployment · Git or Cloud Storage sources · replacing Deployment Manager

GOOGLE-MANAGED TERRAFORM

Infrastructure Manager (Infra Manager) runs Terraform on Google Cloud: you point a **deployment** at Terraform config (local, Cloud Storage or a Git repo) and a **service account**, and it runs plan / apply with Cloud Build, stores state, and keeps **revisions**. **Previews** show changes first. It replaces **Deployment Manager**, which reached end of support on March 31, 2026.

KEY POINTS

- State is managed for you (no backend block needed)
- Pin the Terraform version with `--tf-version-constraint`
- Logs from each run land in Cloud Build / Cloud Storage
- Supported Terraform versions are limited: check the docs before upgrading

DEPLOY, PREVIEW, INSPECT

```
gcloud services enable config.googleapis.com
DEP=projects/shop-prod-4821/locations/us-central1/deployments/shop-prod
SA_EMAIL=infra@shop-prod-4821.iam.gserviceaccount.com
SA=projects/shop-prod-4821/serviceAccounts/$SA_EMAIL

gcloud infra-manager previews create \
    projects/shop-prod-4821/locations/us-central1/previews/shop-pr-42 \
    --service-account=$SA --local-source=./infra \
    --input-values=project=shop-prod-4821,image=api:1.5.0
gcloud infra-manager previews export \
    projects/shop-prod-4821/locations/us-central1/previews/shop-pr-42

gcloud infra-manager deployments apply $DEP \
    --service-account=$SA \
    --git-source-repo=https://github.com/acme/shop-infra \
    --git-source-directory=envs/prod --git-source-ref=main \
    --input-values=project=shop-prod-4821,image=api:1.5.0

gcloud infra-manager deployments describe $DEP
gcloud infra-manager revisions list --deployment=$DEP
gcloud infra-manager resources list --revision=$DEP/revisions/r-3
gcloud infra-manager deployments delete $DEP
```

CODE EXPLAINED

- → The Config API powers Infrastructure Manager
- → The preview produces a plan you can review
- → The deployment pulls Terraform from a Git path and ref
- → Each apply creates a revision with its resources

AWS / AZURE EQUIVALENT

- ↔ **AWS** CloudFormation stacks, change sets (aws-34)
- ↔ **Azure** Deployment stacks, what-if (az-34)

COST CHECK

- $ Infra Manager bills for the Cloud Build minutes and storage it uses
- $ No separate fee for state storage beyond that
- $ Previews are cheap insurance before large changes

CONCEPTS

CONCEPT	MEANING
Deployment	One Terraform root + state
Revision	Result of one apply
Preview	Plan without applying
Service account	Identity that creates resources
Source	Local, GCS or Git

DEPLOYMENT MANAGER → INFRA MANAGER

DEPLOYMENT MANAGER	INFRA MANAGER
YAML / Jinja / Python templates	Terraform HCL
gcloud deployment-manager	gcloud infra-manager
End of support Mar 31, 2026	Supported
DM convert tool	Migrate configs to Terraform

DO & DON'T

- ✓ **Do** Give each deployment its own least-privilege service account
- ✓ **Do** Preview before every apply in production
- ✗ **Don't** Keep running new work on Deployment Manager
- ✗ **Don't** Upgrade Terraform syntax past the versions Infra Manager supports

LOCAL TESTING

- ⌂ `terraform plan` locally against the same config
- ⌂ tflint and Checkov on the module
- ⌂ Previews from pull requests in CI

PRODUCTION CHECKLIST

- ☐ Deployments sourced from Git refs
- ☐ Previews reviewed before applies
- ☐ Revisions retained for audit and rollback

Config Connector & Choosing IaC

Config Connector (Kubernetes resources for Google Cloud) · Config Controller · exporting existing resources · Pulumi pointer · choosing an IaC tool · decision table

KUBERNETES-STYLE INFRASTRUCTURE

Config Connector is a Kubernetes add-on that manages Google Cloud resources as Kubernetes objects (`StorageBucket`, `SQLInstance`, `IAMPolicyMember`…), reconciling continuously. **Config Controller** hosts it as a managed service with Config Sync and Policy Controller. `gcloud resource-config bulk-export` exports existing resources to Terraform or KRM. Choose the tool your team can review and operate.

KEY POINTS

- Config Connector runs on GKE (add-on) or Config Controller
- Resources reconcile: manual changes are reverted
- Pulumi offers general-purpose languages with the Google Cloud provider
- Terraform has the widest ecosystem and examples

CONFIG CONNECTOR RESOURCES

```yaml
apiVersion: storage.cnrm.cloud.google.com/v1beta1
kind: StorageBucket
metadata:
  name: acme-uploads-prod
  namespace: shop
  annotations:
    cnrm.cloud.google.com/project-id: shop-prod-4821
spec:
  location: US-CENTRAL1
  uniformBucketLevelAccess: true
  publicAccessPrevention: enforced
  versioning: { enabled: true }
  lifecycleRule:
    - action: { type: Delete }
      condition: { age: 365 }
---
apiVersion: iam.cnrm.cloud.google.com/v1beta1
kind: IAMPolicyMember
metadata: { name: api-uploads, namespace: shop }
spec:
  member: serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com
  role: roles/storage.objectUser
  resourceRef:
    kind: StorageBucket
    name: acme-uploads-prod
```

CODE EXPLAINED

- → The project annotation picks where the bucket is created
- → Config Connector keeps the bucket matching this spec
- → `IAMPolicyMember` adds one binding (non-authoritative)
- → Bulk export turns existing resources into Terraform files

AWS / AZURE EQUIVALENT

- ↔ **AWS** CDK, AWS Controllers for Kubernetes (aws-35)
- ↔ **Azure** azd, Azure Service Operator (az-35)

APPLY AND EXPORT

```
kubectl apply -f bucket.yaml
kubectl -n shop get storagebuckets
kubectl -n shop describe storagebucket acme-uploads-prod

gcloud resource-config bulk-export --project=shop-prod-4821 \
    --resource-format=terraform --path=./exported
```

COST CHECK

- $ Config Connector is free; you pay for the GKE cluster
- $ Config Controller bills per instance-hour
- $ Pick one tool per resource to avoid fighting reconcilers

CHOOSE AN IAC TOOL

SITUATION	USE
Most teams, multi-cloud	Terraform
Managed Terraform runs on Google Cloud	Infrastructure Manager
Kubernetes / GitOps teams	Config Connector / Config Controller
Code in TypeScript / Python / Go	Pulumi
Legacy Deployment Manager	Migrate to Terraform / Infra Manager

TRADE-OFFS

TOOL	MODEL
Terraform	Plan / apply on demand
Infra Manager	Managed plan / apply
Config Connector	Continuous reconcile
Pulumi	Programs + state service

DO & DON'T

- ✓ **Do** Pick one primary tool and stick to it
- ✓ **Do** Export existing resources instead of rewriting by hand
- ✗ **Don't** Manage the same resource with two tools
- ✗ **Don't** Expect Config Connector to stay quiet about manual console edits (it reverts them)

LOCAL TESTING

- ⌂ kind + Config Connector for experiments (with a dev project)
- ⌂ `kubectl apply --dry-run=server` validates against CRDs
- ⌂ Terraform plans locally with impersonated ADC

PRODUCTION CHECKLIST

- ☐ One tool of record per resource
- ☐ GitOps or CI applies only
- ☐ Drift detected and reconciled automatically

Cloud Monitoring

metrics & resource types · alerting policies · notification channels · uptime checks · dashboards · Managed Service for Prometheus & PromQL

METRICS AND ALERTS

Cloud Monitoring collects metrics from Google Cloud services automatically, from VMs via the **Ops Agent**, and from apps via OpenTelemetry or **Managed Service for Prometheus**. **Alerting policies** fire on metric conditions (MQL is deprecated; use **PromQL** or filter-based conditions) and notify **channels** (email, Slack, PagerDuty, Pub/Sub). **Uptime checks** probe endpoints from around the world.

KEY POINTS

- Metrics scopes let one project view metrics from many projects
- SLO monitoring with burn-rate alerts is built in
- Ops Agent collects VM logs and metrics
- Dashboards can be JSON-defined and version-controlled

ALERT POLICY, CHANNEL, UPTIME CHECK

```
gcloud beta monitoring channels create --display-name="On-call email" \
    --type=email --channel-labels=email_address=oncall@acme.com
gcloud monitoring policies create --policy-from-file=api-5xx.yaml
gcloud monitoring uptime create shop-home --resource-type=uptime-url \
    --resource-labels=host=shop.acme.com,project_id=shop-prod-4821 \
    --path=/healthz --protocol=https --period=1
gcloud monitoring dashboards create --config-from-file=dashboard.json
```

CODE EXPLAINED

- The channel delivers alert notifications by email
- The PromQL condition fires when 5xx exceeds 2% for 5 minutes
- Documentation links the runbook in every notification
- Uptime checks probe `/healthz` every minute from several regions

API-5XX.YAML

```
displayName: "API 5xx rate above 2%"
combiner: OR
conditions:
  - displayName: "5xx ratio"
    conditionPrometheusQueryLanguage:
      query: |
        sum(rate(run_googleapis_com:request_count{
          service_name="api",response_code_class="5xx"}[5m]))
        /
        sum(rate(run_googleapis_com:request_count{service_name="api"}[5m]))
        > 0.02
      duration: 300s
      evaluationInterval: 60s
notificationChannels:
  - projects/shop-prod-4821/notificationChannels/1234567890
documentation:
  content: "Runbook: https://wiki.acme.com/runbooks/api-5xx"
alertStrategy:
  autoClose: 1800s
```

AWS / AZURE EQUIVALENT

- ↔ **AWS** CloudWatch metrics, alarms, Synthetics (aws-37)
- ↔ **Azure** Azure Monitor metrics, alerts, action groups (az-37)

COST CHECK

- $ Google Cloud service metrics are free; custom / Prometheus samples bill per volume
- $ Alerting bills per condition and time series scanned
- $ High-cardinality labels multiply metric costs

METRIC SOURCES

SOURCE	EXAMPLE
Google services	run.googleapis.com/request_count
Ops Agent	VM memory, disk, app logs
Managed Prometheus	GKE / app Prometheus metrics
OpenTelemetry	Custom app metrics
Log-based metrics	Counts from logs (sheet 38)

ALERT BUILDING BLOCKS

PIECE	ROLE
Condition	PromQL / threshold / absence
Duration	How long before firing
Channel	Where notifications go
Documentation	Runbook text
Snooze	Planned maintenance quiet

DO & DON'T

- ✓ **Do** Alert on user-facing symptoms (errors, latency, SLO burn)
- ✓ **Do** Add a runbook link to every policy
- ✗ **Don't** Write new alerts in MQL (deprecated)
- ✗ **Don't** Page on every CPU spike

LOCAL TESTING

- Run Prometheus + Grafana locally to draft PromQL
- Test policies in a dev project with low thresholds
- Export dashboards as JSON and commit them

PRODUCTION CHECKLIST

- ☐ SLOs with burn-rate alerts on key services
- ☐ Uptime checks on public endpoints
- ☐ Policies and dashboards as code

Cloud Logging & Trace

Log Router & sinks · log buckets & retention · Logging query language · log-based metrics · Cloud Trace with OpenTelemetry · Error Reporting

LOGS AND TRACES

Cloud Logging ingests logs from services, agents and apps (structured JSON on stdout works on Cloud Run and GKE). The **Log Router** sends entries through **sinks** to **log buckets**, BigQuery, Cloud Storage or Pub/Sub. Query with the **Logging query language** or **Log Analytics** (SQL). **Cloud Trace** receives OpenTelemetry spans, and **Error Reporting** groups exceptions.

KEY POINTS

- `_Required` bucket keeps audit logs 400 days; `_Default` keeps 30 days by default
- Exclusion filters drop noisy logs before ingestion charges
- Write JSON with `severity` and `message` fields for structured logs
- Trace context propagates via `traceparent` headers

QUERY, ROUTE, RETAIN

```
gcloud logging read 'resource.type="cloud_run_revision"
  AND resource.labels.service_name="api" AND severity>=ERROR' \
  --freshness=1h --limit=20 --format="value(timestamp,jsonPayload.message)"

gcloud logging buckets update _Default --location=global --retention-days=90
gcloud logging sinks create audit-to-bq \
    bigquery.googleapis.com/projects/sec-logs-1111/datasets/audit \
    --log-filter='logName:"cloudaudit.googleapis.com"' \
    --organization=123456789012 --include-children
gcloud logging sinks update _Default \
    --add-exclusion=name=no-health,filter='httpRequest.requestUrl:"/healthz"'
gcloud logging metrics create checkout_errors \
    --log-filter='jsonPayload.event="checkout_failed"'
```

CODE EXPLAINED

→ The query finds recent errors from one Cloud Run service

→ The org-level sink copies all audit logs to BigQuery

→ The exclusion drops health-check request logs

→ JSON lines become structured entries with severity

STRUCTURED LOGS AND TRACES (PYTHON)

```
import json, sys
def log(severity, message, **fields):
    print(json.dumps({"severity": severity, "message": message, **fields}),
          file=sys.stdout, flush=True)
log("ERROR", "checkout failed", event="checkout_failed", order="o-1001")

from opentelemetry import trace
tracer = trace.get_tracer("shop.api")
with tracer.start_as_current_span("charge-card") as span:
    span.set_attribute("order.id", "o-1001")
    charge()
```

AWS / AZURE EQUIVALENT

↔	**AWS**	CloudWatch Logs, Logs Insights, X-Ray (aws-38)
↔	**Azure**	Log Analytics, Application Insights (az-38)

COST CHECK

$ Ingestion per GiB after 50 GiB per project per month free

$ Retention beyond 30 days bills per GiB-month

$ Exclusions and sampling cut the biggest costs

LIMITS & QUOTAS

≤ Log entry size up to 256 KB

≤ Default `_Default` retention 30 days (configurable up to 3,650)

≤ Write quotas per project per minute (batch writes)

SINK DESTINATIONS

DESTINATION	USE
`Log bucket`	Search, Log Analytics
`BigQuery`	SQL analysis, long retention
`Cloud Storage`	Cheap archive
`Pub/Sub`	Stream to SIEM / tools
`Another project`	Central logging

QUERY LANGUAGE

EXAMPLE	MATCHES
`severity>=ERROR`	Errors and above
`resource.type="k8s_container"`	GKE containers
`jsonPayload.user="u1"`	Structured field
`textPayload:"timeout"`	Substring
`timestamp>="2026-11-12T00:00:00Z"`	Time range

DO & DON'T

✓ **Do** Log structured JSON with severity

✓ **Do** Centralize audit logs with an org-level sink

✗ **Don't** Log secrets, tokens or full request bodies

✗ **Don't** Keep debug logging on in production without exclusions

LOCAL TESTING

⌂ Print the same JSON locally; read with `jq`

⌂ Jaeger in Docker receives OTLP spans locally

⌂ `gcloud logging tail` streams live logs (beta)

PRODUCTION CHECKLIST

☐ Retention set per bucket; audit logs centralized

☐ Exclusions for noisy logs

☐ Traces sampled and linked to logs

Deploying to Google Cloud

`google-github-actions/auth` (WIF) · Cloud Build & `cloudbuild.yaml` · Cloud Deploy pipelines · Cloud Run canary via traffic splitting · Binary Authorization · labels strategy

CI/CD INTO GOOGLE CLOUD

From GitHub Actions, authenticate with `google-github-actions/auth` using **Workload Identity Federation** (sheet 07), then build and deploy. **Cloud Build** runs build steps defined in `cloudbuild.yaml` with triggers from GitHub or other repos. **Cloud Deploy** promotes releases through targets (dev → staging → prod) with approvals and canary strategies for Cloud Run and GKE.

KEY POINTS

- `id-token: write` permission is required for WIF in GitHub Actions
- Build once, promote the same image digest through environments
- Cloud Deploy supports canary percentages and verification jobs
- Labels like `app`, `env`, `team` tie deployments to costs

.GITHUB/WORKFLOWS/DEPLOY.YML

```yaml
on: { push: { branches: [main] } }
permissions: { contents: read, id-token: write }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    env:
      IMG: us-central1-docker.pkg.dev/shop-prod-4821/apps/api:${{ github.sha }}
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v3
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: deploy@shop-prod-4821.iam.gserviceaccount.com
      - uses: google-github-actions/setup-gcloud@v2
      - run: gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
      - run: docker build -t "$IMG" . && docker push "$IMG"
      - run: >
          gcloud run deploy api --image="$IMG" --region=us-central1
          --no-traffic --tag=canary
```

CODE EXPLAINED

→ The job gets a GitHub OIDC token and exchanges it via WIF

→ The new revision deploys with no traffic under a `canary` tag

→ Cloud Build builds, pushes and creates a Cloud Deploy release

→ Cloud Deploy then promotes the release through its targets

CLOUDBUILD.YAML

```yaml
steps:
  - name: gcr.io/cloud-builders/docker
    args: [build, -t, "$_IMG:$SHORT_SHA", .]
  - name: gcr.io/cloud-builders/docker
    args: [push, "$_IMG:$SHORT_SHA"]
  - name: gcr.io/google.com/cloudsdktool/cloud-sdk
    entrypoint: gcloud
    args: [deploy, releases, create, "rel-$SHORT_SHA",
           --delivery-pipeline=shop, --region=us-central1,
           "--images=api=$_IMG:$SHORT_SHA"]
substitutions:
  _IMG: us-central1-docker.pkg.dev/$PROJECT_ID/apps/api
```

AWS / AZURE EQUIVALENT

↔ **AWS** GitHub OIDC role, CodePipeline / CodeDeploy (aws-39)

↔ **Azure** azure/login with OIDC, Azure Pipelines (az-39)

COST CHECK

$ Cloud Build: free build minutes each month, then per build-minute by machine type

$ Cloud Deploy: per active delivery pipeline (first one free)

$ GitHub-hosted runners bill on GitHub, not Google Cloud

SECURITY CHECK

⚠ WIF provider condition pinned to the repository and branch / environment

⚠ Deploy service account with only run / artifact roles

⚠ Binary Authorization to block unsigned or unscanned images

PIPELINE OPTIONS

OPTION	USE
GitHub Actions + WIF	Code already on GitHub
Cloud Build triggers	Builds inside Google Cloud
Cloud Deploy	Promotion, approvals, canary
Config Sync / Argo CD	GitOps for GKE
Infra Manager	Terraform deployments

ROLLOUT STRATEGIES

STRATEGY	HOW
Cloud Run traffic split	Tags + percentages
Cloud Deploy canary	10% → 50% → 100% phases
GKE rolling update	Deployment strategy
MIG canary	Second template at %
Blue / green	Switch LB backend or traffic

DO & DON'T

✓ **Do** Build once and deploy by digest everywhere

✓ **Do** Gate production with GitHub environments or Cloud Deploy approvals

✗ **Don't** Store service account JSON keys as GitHub secrets

✗ **Don't** Rebuild images separately for each environment

LOCAL TESTING

⌂ `act` runs GitHub workflows locally (no real OIDC)

⌂ `cloud-build-local` is retired: test steps with Docker directly

⌂ Skaffold renders and tests Cloud Deploy manifests

PRODUCTION CHECKLIST

☐ Keyless CI with WIF and narrow conditions

☐ Canary + automatic rollback on alerts

☐ Deploy history auditable (Cloud Deploy / audit logs)

Agent Platform (Vertex AI) & Gemini

Gemini models on Gemini Enterprise Agent Platform · Google Gen AI SDK (`google-genai`) · `generate_content` & streaming · grounding with Google Search · RAG Engine / Vertex AI Search · Agent Builder & ADK

GENERATIVE AI ON GOOGLE CLOUD

Gemini Enterprise Agent Platform (the April 2026 rename of **Vertex AI**) serves **Gemini** models and partner / open models from **Model Garden**. Call them with the **Google Gen AI SDK** (`google-genai`) using ADC and IAM, no API keys. Features include streaming, structured output, function calling, **grounding** with Google Search or your data (**RAG Engine**, Vertex AI Search) and **Agent Builder** with the Agent Development Kit (ADK).

KEY POINTS

- The `global` location routes to available capacity; regional locations keep data in a region
- Model IDs change often: check the model versions page before pinning
- Older Gemini 2.5 models are being retired; plan migrations
- Provisioned Throughput reserves capacity for production

GEMINI WITH THE GEN AI SDK (PYTHON)

```python
from google import genai
from google.genai import types

client = genai.Client(vertexai=True, project="shop-prod-4821",
                      location="global")

resp = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="Summarize this support ticket in two sentences: ...",
    config=types.GenerateContentConfig(temperature=0.2,
                                       max_output_tokens=300))
print(resp.text, resp.usage_metadata.total_token_count)

for chunk in client.models.generate_content_stream(
        model="gemini-3.8-flash", contents="Write a product blurb"):
    print(chunk.text, end="")

grounded = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="What changed in the latest Cloud Run release notes?",
    config=types.GenerateContentConfig(
        tools=[types.Tool(google_search=types.GoogleSearch())]))
print(grounded.text)
```

CODE EXPLAINED

- → `vertexai=True` uses ADC and the Agent Platform endpoint
- → Usage metadata reports tokens for cost tracking
- → Streaming prints text as it is generated
- → The Google Search tool grounds answers in fresh web results

AWS / AZURE EQUIVALENT

- ↔ **AWS** Amazon Bedrock (aws-40)
- ↔ **Azure** Microsoft Foundry & Azure OpenAI (az-40)

COST CHECK

- $ Billed per input / output token (rates differ by model and context length)
- $ Batch prediction and context caching cut costs
- $ Grounding with Google Search bills per grounded request beyond free allowances

SECURITY CHECK

- ⚠ Grant `roles/aiplatform.user` to the app's service account only
- ⚠ Use VPC Service Controls around AI APIs for sensitive data
- ⚠ Never send secrets or unneeded personal data in prompts

SETUP

```
gcloud services enable aiplatform.googleapis.com
gcloud projects add-iam-policy-binding shop-prod-4821 \
    --member="serviceAccount:api@shop-prod-4821.iam.gserviceaccount.com" \
    --role="roles/aiplatform.user"
```

LIMITS & QUOTAS

- ≤ Quotas are per model and region (or dynamic shared quota on `global`)
- ≤ 429 errors mean quota / capacity: retry with backoff
- ≤ Context window and output tokens are limited per model

BUILDING BLOCKS

FEATURE	USE
Gemini models	Text, code, images, audio, video
Model Garden	Partner and open models (Claude, Llama…)
RAG Engine / Vertex AI Search	Ground on your documents
Agent Builder + ADK	Build and deploy agents
Agent Engine	Managed agent runtime
Evaluation	Compare prompts / models

SDK BASICS

CALL	DOES
`models.generate_content`	One response
`models.generate_content_stream`	Streaming
`models.count_tokens`	Estimate tokens
`chats.create`	Multi-turn chat
`caches.create`	Context caching

DO & DON'T

- ✓ **Do** Use structured output (`response_schema`) for machine-read results
- ✓ **Do** Log token usage per request for cost attribution
- ✗ **Don't** Hard-code a model ID without a plan to update it
- ✗ **Don't** Use API keys in server apps (use ADC / service accounts)

LOCAL TESTING

- ⌂ Same SDK with ADC from `gcloud auth application-default login`
- ⌂ Agent Studio (formerly Vertex AI Studio) to draft prompts
- ⌂ Mock the client in unit tests; run evals separately

PRODUCTION CHECKLIST

- ☐ Dedicated SA with aiplatform.user
- ☐ Retries with backoff for 429s; quota monitored
- ☐ Prompt / response logging policy reviewed for privacy

AI & ML Services

custom training & endpoints · Model Garden · Vision API · Speech-to-Text & Text-to-Speech · Document AI · Translation · Gemini Code Assist

PREBUILT AND CUSTOM ML

Google Cloud offers **prebuilt APIs** for common tasks (Vision, Speech-to-Text, Text-to-Speech, Translation, Natural Language, Video Intelligence), **Document AI** for parsing forms, invoices and IDs, and the Agent Platform (Vertex AI) for **custom training**, **endpoints**, pipelines and model registry. **Gemini Code Assist** brings AI help into IDEs and the console.

KEY POINTS

- Try Gemini first: many vision, OCR and classification tasks no longer need a dedicated API
- Document AI processors are created per type (OCR, Form, Invoice…)
- Custom training runs your container on managed infrastructure (GPUs / TPUs)
- BigQuery ML trains models with SQL

PREBUILT APIS (PYTHON)

```
from google.cloud import vision, translate_v3, documentai

img = vision.Image(source=vision.ImageSource(
    image_uri="gs://acme-media-prod/2026/receipt.jpg"))
labels = vision.ImageAnnotatorClient().label_detection(image=img)
print([l.description for l in labels.label_annotations])

tr = translate_v3.TranslationServiceClient()
out = tr.translate_text(parent="projects/shop-prod-4821/locations/global",
    contents=["Your order has shipped"], target_language_code="es")
print(out.translations[0].translated_text)

doc_client = documentai.DocumentProcessorServiceClient()
name = doc_client.processor_path("shop-prod-4821", "us", "a1b2c3d4e5f6")
with open("invoice.pdf", "rb") as f:
    raw = documentai.RawDocument(content=f.read(),
                                 mime_type="application/pdf")
result = doc_client.process_document(
    request=documentai.ProcessRequest(name=name, raw_document=raw))
for e in result.document.entities:
    print(e.type_, e.mention_text)
```

CODE EXPLAINED

- → Vision labels an image straight from Cloud Storage
- → Translation uses the `global` location
- → The invoice processor returns typed entities (totals, dates)
- → Custom jobs run your training container on managed VMs

AWS / AZURE EQUIVALENT

- ↔ **AWS** Rekognition, Textract, Transcribe, SageMaker (aws-41)
- ↔ **Azure** Foundry Tools, Document Intelligence, Azure ML (az-41)

CUSTOM TRAINING AND ENDPOINT

```
TRAIN=us-central1-docker.pkg.dev/shop-prod-4821/ml/train:1.0
POOL=machine-type=n1-standard-8,replica-count=1
gcloud ai custom-jobs create --region=us-central1 --display-name=churn \
    --worker-pool-spec=$POOL,container-image-uri=$TRAIN
gcloud ai endpoints create --region=us-central1 --display-name=churn
```

COST CHECK

- $ Prebuilt APIs bill per unit (image, minute, character, page) with free monthly tiers
- $ Endpoints bill per node-hour while deployed, even idle
- $ Training bills per machine / accelerator hour

PICK A SERVICE

TASK	SERVICE
Image labels / OCR	Vision API or Gemini
Forms, invoices, IDs	Document AI
Transcribe audio	Speech-to-Text
Read text aloud	Text-to-Speech
Translate	Translation API
Custom model	Agent Platform training
SQL-based ML	BigQuery ML

ML PLATFORM PIECES

PIECE	DOES
Workbench	Managed notebooks
Custom training	Run training containers
Model Registry	Version models
Endpoints	Online prediction
Batch prediction	Offline scoring
Pipelines	ML workflows

DO & DON'T

- ✓ **Do** Start with a prebuilt API or Gemini before custom training
- ✓ **Do** Undeploy idle endpoints
- ✗ **Don't** Send sensitive documents without checking data residency
- ✗ **Don't** Train custom models for tasks a prebuilt API already does well

LOCAL TESTING

- ⌂ Client libraries work locally with ADC
- ⌂ Small sample files for quick API checks
- ⌂ Train locally in the same container before submitting jobs

PRODUCTION CHECKLIST

- ☐ Processors and endpoints in IaC
- ☐ Monitoring on prediction latency and errors
- ☐ Cost alerts on per-unit API usage

Sheet 42

Quick Reference — gcloud & Service Picker

gcloud config / auth · projects / iam · compute · storage · run / functions · container / artifacts · logging · bq · Need → Google Cloud service → AWS → Azure (with sheet numbers) · default quotas worth knowing · pricing model per service

SETUP & IDENTITY

COMMAND	DOES	SHEET #
gcloud init / auth login	Sign in CLI	03
gcloud auth application-default login	Credentials for code	04
gcloud config configurations activate c	Switch config	03
gcloud projects add-iam-policy-binding	Grant role	05
gcloud iam service-accounts create	Service account	07
gcloud secrets versions access latest --secret=s	Read secret	08
gcloud billing budgets create	Budget alert	02

COMPUTE & SERVERLESS

COMMAND	DOES	SHEET #
gcloud compute instances create	VM	15
gcloud compute ssh vm --tunnel-through-iap	SSH, no public IP	15
gcloud compute instance-groups managed create	MIG	18
gcloud run deploy --source=.	Cloud Run	20
gcloud run services update-traffic	Canary	20
gcloud run jobs execute	Run job	21
gcloud functions deploy --gen2	Function	21

CONTAINERS

COMMAND	DOES	SH #
gcloud auth configure-docker REGION-docker.pkg.dev	Registry login	25
gcloud builds submit --tag=IMG	Build in cloud	25
gcloud container clusters create-auto	GKE Autopilot	26
gcloud container clusters get-credentials	kubectl access	26

STORAGE & DATA

COMMAND	DOES	S #
gcloud storage cp / rsync / ls / rm	Objects	2
gcloud storage sign-url	Temporary URL	2
gcloud sql instances create	Database	3
gcloud pubsub topics publish	Message	2
bq query --use_legacy_sql=false	SQL analytics	3

IAC & OPS

COMMAND	DOES
terraform plan / apply	Terraform
gcloud infra-manager deployments apply	Managed Terraform
gcloud monitoring policies create	Alert
gcloud logging read FILTER	Query logs
gcloud deploy releases create	Cloud Deploy

SERVICE PICKER

NEED	GOOGLE CLOUD	AWS	AZURE	SHEET #
People sign-in	Cloud Identity	IAM Identity Center	Entra ID	05
Workload identity	Service account	IAM role	Managed identity	07
Secrets	Secret Manager	Secrets Manager	Key Vault	08
Private network	VPC (global)	VPC	VNet	10
CDN	Cloud CDN	CloudFront	Front Door	14
VMs / scaling	Compute Engine / MIG	EC2 / ASG	VMs / VMSS	15, 18
Serverless containers	Cloud Run	Fargate / App Runner	Container Apps	20
Functions	Cloud Run functions	Lambda	Azure Functions	21
Kubernetes	GKE	EKS	AKS	26
Object storage	Cloud Storage	S3	Blob Storage	28
Relational	Cloud SQL / AlloyDB	RDS / Aurora	Azure SQL / PostgreSQL	31
NoSQL	Firestore	DynamoDB	Cosmos DB	32
Queue / topic	Pub/Sub	SQS / SNS	Service Bus	24
Native IaC	Infrastructure Manager	CloudFormation / CDK	Bicep	35
Monitoring	Cloud Monitoring	CloudWatch	Azure Monitor	37
Generative AI	Agent Platform (Vertex AI)	Bedrock	Microsoft Foundry	40

LIMITS WORTH KNOWING

LIMIT	DEFAULT
Cloud Run request timeout	60 min max
Cloud Run concurrency	1,000 max per instance
Pub/Sub message	10 MB
Firestore document	1 MiB
Cloud Storage object	5 TiB
Secret payload	64 KiB
Log entry	256 KB
Firewall rules per project	100 (adjustable)

PRICING MODELS

SERVICE	BILLED BY
Compute Engine	Per second vCPU + memory + disks
Cloud Run	vCPU-s + GiB-s + requests
GKE Autopilot	Pod resource requests
Cloud Storage	GB-month + operations + egress
BigQuery	TiB scanned or slot-hours
Cloud NAT	Hourly + per GB
Gemini	Input / output tokens